

SOLIDWORKS DimXpert API (.NET) — Examples

(4/7): Get_DimXpert_Tolerance1_Example_VB-
Get_DimXpert_Tolerance2_Example_VB.NET

SOLIDWORKS API Documentation (converted from CHM)

Get DimXpert Tolerance1 Example (VBA)

This example shows how to build a part and get attributes for the following DimXpert annotations:

- * Circularity geometric tolerance
- * Cylindricity geometric tolerance
- * Countersink angle dimension tolerance
- * Countersink diameter dimension tolerance
- * Flatness geometric tolerance
- * Surface profile geometric tolerance

```
' -----  
' Preconditions:  
' 1. Open: public_documents\samples\tutorial\api\cover_with_geometric_tolerances.sldprt.  
' 2. Click Geometric Tolerance on the DimXpert toolbar.  
'   In the Geometric Tolerance Properties dialog:  
'   a. Select Cylindricity from the Symbol dropdown.  
'   b. In the second row, select Profile of Surface from the Symbol dropdown  
'      and type a tolerance of 0.5 next to it.  
'   c. Click the outer base cylinder and click again outside the part  
'      to place the annotation.  
'   d. Click OK to close the Geometric Tolerance Properties dialog.  
' 3. In the DimXpertManager tab of the Management Panel,  
'    expand all of the nodes in the tree.  
' 4. Observe the following Dimxpert annotations:  
'    Flatness1, Circularity1, Diameter1,  
'    CounterSinkAngle1, CounterSinkDiameter1, Cylindricity1,  
'    and Surface Profile1  
' 5. Ensure that the latest SOLIDWORKS DimXpert type library is loaded  
'    (click Tools > References).  
' 6. Ensure that the Microsoft Scripting Runtime library is loaded  
'    (click Tools > References).  
'  
' Postconditions:  
' 1. Writes the output to c:\temp\dimXpertInfo.txt.  
' 2. Inspect the Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
' -----
```

Option Explicit

Dim strs As New Collection

Dim dimXpertPart As SwDimXpert.dimXpertPart

Sub Main()

Dim swapp As SldWorks.SldWorks

```

    Set swapp = Application.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Set swModelDoc = swapp.ActiveDoc
If swModelDoc Is Nothing Then
    Exit Sub
End If
Dim f As New FileSystemObject
Dim textStr As TextStream
Set textStr = f.CreateTextFile("C:\temp\dimXpertInfo.txt", True)
If textStr Is Nothing Then
    Debug.Print "Error creating temp file."
    Exit Sub
End If

Call log("Starting DimXpert log...", textStr)
Call retrieve_info_text(swapp, textStr)
textStr.Close
End Sub

Private Sub log(text As String, textStr As TextStream)
    Debug.Print text
    textStr.WriteLine (text)
End Sub

Private Sub retrieve_info_text(swapp As SldWorks.SldWorks, textStr As TextStream)
    Dim dimXpertMgr As SldWorks.DimXpertManager

    Set dimXpertMgr =
swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
True)
    Call log("Model: " & swapp.IActiveDoc2.GetPathName, textStr)
    Dim dimXpertPartObj As dimXpertPart
    Set dimXpertPartObj = dimXpertMgr.dimXpertPart

    Set dimXpertPart = dimXpertPartObj

    Dim vAnnotations As Variant

    vAnnotations = dimXpertPart.GetAnnotations()

    Call log("-----", textStr)
    Call log("Annotations...", textStr)
    Call log("-----", textStr)

    Dim annotationTemp As DimXpertAnnotation

```

```

Dim annotationIndex As Long
For annotationIndex = 0 To UBound(vAnnotations)
    Set annotationTemp = vAnnotations(annotationIndex)

    Dim AnnotationDataText As Collection

    Set AnnotationDataText = AnnotationData(annotationTemp)
    Dim annotationTextIndex As Long
    For annotationTextIndex = 1 To AnnotationDataText.Count
        Call log(AnnotationDataText(annotationTextIndex), textStr)
    Next
Next

```

End Sub

```

Public Function AnnotationData(annotation As DimXpertAnnotation) As Collection
    Dim annoType As Long

    'general info
    Call GeneralInfo(annotation)
    annoType = annotation.Type
    If annoType = swDimXpertDatum Then
        Call DatumData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Position Then
        Call PositionData(annotation)

    ElseIf annoType = swDimXpertGeoTol_CompositePosition Then
        Call CompositePositionData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Symmetry Then
        Call SymmetryData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Concentricity Then
        Call ConcentricityData(annotation)

    ElseIf annoType = swDimXpertGeoTol_LineProfile Then
        Call LineProfileData(annotation)

    ElseIf annoType = swDimXpertGeoTol_CompositeLineProfile Then
        Call CompositeLineProfileData(annotation)

    ElseIf annoType = swDimXpertGeoTol_SurfaceProfile Then
        Call SurfaceProfileData(annotation)
    End If
End Function

```

```

ElseIf annoType = swDimXpertGeoTol_CompositeSurfaceProfile Then
    Call CompositeSurfaceProfileData(annotation)

ElseIf annoType = swDimXpertGeoTol_Angularity Or annoType = swDimXpertGeoTol_Parallelism
Or annoType = swDimXpertGeoTol_Perpendicularity Then
    Call OrientationData(annotation)

ElseIf annoType = swDimXpertGeoTol_TotalRunout Then
    Call TotalRunoutData(annotation)

ElseIf annoType = swDimXpertGeoTol_CircularRunout Then
    Call CircularRunoutData(annotation)

ElseIf annoType = swDimXpertGeoTol_Flatness Then
    Call FlatnessData(annotation)

ElseIf annoType = swDimXpertGeoTol_Circularity Then
    Call CircularityData(annotation)

ElseIf annoType = swDimXpertGeoTol_Cylindricity Then
    Call CylindricityData(annotation)

ElseIf annoType = swDimXpertGeoTol_Straightness Then
    Call StraightnessData(annotation)
ElseIf annoType = swDimXpertGeoTol_Tangency Then
    Call TangencyData(annotation)
Else ' any of the dimension tolerance types
    Call DimensionToleranceData(annotation)

End If

Set AnnotationData = strs
End Function
Private Sub Clear(strs As Collection)
    Dim n As Long
    strs.Remove (strs.Count)
    If Not strs.Count = 0 Then
        Call Clear(strs)
    End If
End Sub
Private Sub GeneralInfo(annotation As DimXpertAnnotation)
    Dim annoType As String
    Dim modelObj As Object
    Dim modelFeature As SldWorks.feature

```

```

If Not strs.Count = 0 Then
    Call Clear(strs)
End If

strs.Add ("")
strs.Add ("Name: " + annotation.Name)

annoType = annotationTypeNameFromObject(annotation)
strs.Add ("Type: " + annoType)
strs.Add ("Display Entity: " + DisplayEntity(annotation))

Set modelObj = annotation.GetModelFeature
Set modelFeature = modelObj
If Not (modelFeature Is Nothing) Then
    strs.Add ("ModelFeature: " + modelFeature.Name + " (" + modelFeature.GetTypeName2() +
    ")")
End If
End Sub

Private Sub DatumData(annotation As DimXpertDatum)
    ' the datum letter
    strs.Add ("")
    strs.Add ("Datum Letter: " + annotation.Identifier)

End Sub

Private Sub PositionData(annotation As DimXpertPositionTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean

    strs.Add ("")
    strs.Add ("Position Tolerance Compartment:")

    ' the shape of the tolerance zone
    strs.Add (" Zone Type: " + PositionZoneType(annotation.zoneType))

    ' the zone vector if the tolerance zone is planar
    If annotation.zoneType = swDimXpertPositionZoneType_PlanarPosition Then
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        strs.Add (" Direction: " + FormatVector(I, J, K))
    End If

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the material condition modifier applied to feature

```

```
strs.Add (" Modifier: " + mcmStr(annotation.modifier))
```

```
' the projected tolerance zone when specified  
boolstatus = annotation.GetProjectedZone(enabled, value)  
Call FormatProjectedZone(enabled, value)
```

```
' the datum reference frame  
Call DatumsStr(annotation)
```

```
End Sub
```

```
Private Sub CompositePositionData(annotation As DimXpertCompositePositionTolerance)
```

```
Dim I As Double, J As Double, K As Double  
Dim enabled As Boolean, value As Double  
Dim boolstatus As Boolean
```

```
strs.Add ("")  
strs.Add ("Composite Position Tolerance Compartment")
```

```
' the shape of the tolerance zone  
strs.Add (" Zone Type: " + PositionZoneType(annotation.zoneType))
```

```
' the zone vector when the zone is planar  
If annotation.zoneType = swDimXpertPositionZoneType_PlanarPosition Then  
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)  
    strs.Add (" Direction: " + FormatVector(I, J, K))  
End If
```

```
' the tolerance value  
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

```
' the material condition modifier  
strs.Add (" Modifier: " + mcmStr(annotation.modifier))
```

```
' the projected tolerance zone when specified  
boolstatus = annotation.GetProjectedZone(enabled, value)  
Call FormatProjectedZone(enabled, value)
```

```
' the datum reference frame for the pattern location  
Call DatumsStr(annotation)
```

```
' the datum reference frame for the feature to feature location  
strs.Add ("Composite datums:")  
strs.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))  
strs.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))
```

```
    strs.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))
```

```
End Sub
```

```
Private Sub SymmetryData(annotation As DimXpertSymmetryTolerance)
```

```
    Dim I As Double, J As Double, K As Double
```

```
    Dim boolstatus As Boolean
```

```
    strs.Add ("")
```

```
    strs.Add ("Symmetry Tolerance Compartment")
```

```
    ' the tolerance value
```

```
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

```
    ' the material condition modifier applied to feature
```

```
    strs.Add (" Modifier: " + mcmStr(annotation.modifier))
```

```
    ' the datum reference frame
```

```
    strs.Add ("")
```

```
    Call DatumsStr(annotation)
```

```
    ' the direction of the planar zone
```

```
    strs.Add ("")
```

```
    boolstatus = annotation.GetZoneDirection(I, J, K)
```

```
    strs.Add ("Planar Zone Direction: " + FormatVector(I, J, K))
```

```
End Sub
```

```
Private Sub ConcentricityData(annotation As DimXpertConcentricityTolerance)
```

```
    strs.Add ("")
```

```
    strs.Add ("Concentricity Tolerance Compartment")
```

```
    ' the tolerance value
```

```
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

```
    ' the material condition modifier applied to feature
```

```
    strs.Add (" Modifier: " + mcmStr(annotation.modifier))
```

```
    ' the datum reference frame
```

```
    Call DatumsStr(annotation)
```

```
End Sub
```

```
Private Sub TotalRunoutData(annotation As DimXpertTolerance)
```

```
    strs.Add ("")
```

```
    strs.Add ("Total Runout Tolerance Compartment")
```

```
    ' the tolerance value
```

```
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

```
' the datum reference frame  
Call DatumsStr(annotation)
```

End Sub

```
Private Sub CircularRunoutData(annotation As DimXpertTolerance)  
    strs.Add ("")  
    strs.Add ("Circular Runout Tolerance Compartment")  
  
    ' the tolerance value  
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

```
' the datum reference frame  
Call DatumsStr(annotation)
```

End Sub

```
Private Sub LineProfileData(annotation As DimXpertLineProfileTolerance)  
    Dim I As Double, J As Double, K As Double  
    Dim boolstatus As Boolean  
  
    strs.Add ("")  
    strs.Add ("Line Profile Tolerance Compartment")  
  
    ' the tolerance value  
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))  
  
    ' the outer (outside material) tolerance value  
    strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))  
  
    ' the vector normal to the profile zones  
    strs.Add ("")  
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)  
    strs.Add ("Planar Zone Vector: " + FormatVector(I, J, K))  
  
    ' the datum reference frame  
    Call DatumsStr(annotation)
```

End Sub

```
Private Sub CompositeLineProfileData(annotation As DimXpertCompositeLineProfileTolerance)  
  
    strs.Add ("")  
    strs.Add ("Composite Line Profile Tolerance Compartment")  
  
    ' the tolerance value  
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
```

```

' the outer (outside material) tolerance value
strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))

' the vector normal to the profile zones
strs.Add ("")
boolstatus = annotation.GetPlanarZoneVector(I, J, K)
strs.Add ("Planar Zone Vector: " + FormatVector(I, J, K))

' the datum reference frame
Call DatumsStr(annotation)

' the datum reference frame for the orientation and form
strs.Add ("Composite Datums:")
strs.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
strs.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))
strs.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))

```

End Sub

```
Private Sub SurfaceProfileData(annotation As DimXpertSurfaceProfileTolerance)
```

```

strs.Add ("")
strs.Add ("Surface Profile Tolerance Compartment")

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

' the outer (outside material) tolerance value
strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))

' the datum reference frame
Call DatumsStr(annotation)

```

End Sub

```
Private Sub CompositeSurfaceProfileData(annotation As
DimXpertCompositeSurfaceProfileTolerance)
```

```

strs.Add ("")
strs.Add ("Composite Surface Profile Tolerance Compartment")

' the tolerance value
strs.Add (" Tolerance Upper Tier: " + FormatDouble(annotation.Tolerance))

' the outer tolerance value

```

```

    strs.Add (" Outer Tolerance Upper Tier: " +
FormatDouble(annotation.OuterToleranceValue))

' the datum reference frame for the location
Call DatumsStr(annotation)

' the datum reference frame for the orientation and form
strs.Add ("Composite Datums:")
strs.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
strs.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))
strs.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))
End Sub
Private Sub OrientationData(annotation As DimXpertOrientationTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim annoType As Long
    Dim boolstatus As Boolean
    annoType = annotation.Type
    ' the type or orientation tolerance
    If annoType = swDimXpertGeoTol_Perpendicularity Then
        strs.Add ("Orientation Type: Perpendicularity")
    ElseIf annoType = swDimXpertGeoTol_Parallelism Then
        strs.Add ("Orientation Type: Parallelism")
    ElseIf annoType = swDimXpertGeoTol_Angularity Then
        strs.Add ("Orientation Type: Angularity")
    End If

    strs.Add ("")
    strs.Add ("Orientation Tolerance Compartment:")

    ' the shape of the tolerance zone
    Select Case annotation.zoneType
    Case swDimXpertOrientationZoneType_Cylindrical
        strs.Add (" Zone Type: Cylindrical")
    Case swDimXpertOrientationZoneType_Planar
        strs.Add (" Zone Type: Planar")
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        strs.Add ("Planar Zone Vector: " + FormatVector(I, J, K))
    End Select

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    'material condition modifier applied to feature
    strs.Add (" Modifier: " + mcmStr(annotation.modifier))

```

```

' the projected tolerance zone when specified
boolstatus = annotation.GetProjectedZone(enabled, value)
Call FormatProjectedZone(enabled, value)

' is tangent plane
strs.Add (" IsTangentPlane: " + IIf(annotation.IsTangentPlane, "True", "False"))

' the datum reference frame
Call DatumsStr(annotation)

```

End Sub

```

Private Sub FlatnessData(annotation As DimXpertFlatnessTolerance)
    Dim enabled As Boolean
    Dim length As Double, width As Double
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean

    strs.Add ("")
    strs.Add ("Flatness Tolerance Compartment:")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the per unit area data
    boolstatus = annotation.GetPerUnitArea(enabled, length, width, I, J, K)
    strs.Add (" Per Unit Area: " + IIf(enabled, "True", "False"))
    If enabled Then
        strs.Add (" Per Unit Length: " + FormatDouble(length))
        strs.Add (" Per Unit Width: " + FormatDouble(width))
        strs.Add (" Per Unit Direction: " + FormatVector(I, J, K))
    End If
End Sub

```

End Sub

```

Private Sub CircularityData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Circularity Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

```

End Sub

```

Private Sub CylindricityData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Cylindricity Tolerance Compartment")
End Sub

```

```

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

End Sub

Private Sub StraightnessData(annotation As DimXpertStraightnessTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean
    Dim dist As Double
    Dim boolstatus As Boolean
    'tolerance compartment info
    strs.Add ("")
    strs.Add ("Straightness Tolerance Compartment")

    'type or shape of the tolerance zone
    Select Case annotation.zoneType
    Case swDimXpertStraightnessZoneType_Cylindrical
        strs.Add (" Zone Type: Cylindrical")
    Case swDimXpertStraightnessZoneType_PlanarMedian
        strs.Add (" Zone Type: Planar Median")
    Case swDimXpertStraightnessZoneType_Surface
        strs.Add (" Zone Type: Surface Straightness")
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        strs.Add (" Zone Vector: " + FormatVector(I, J, K))
    End Select

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    'per unit length
    boolstatus = annotation.GetPerUnitLength(enabled, dist)
    strs.Add (" Per Unit Length: " + IIf(enabled, "True", "False"))
    If enabled Then
        strs.Add (" Per Unit Length Distance: " + FormatDouble(dist))
    End If

    ' the material condition modifier
    strs.Add (" Modifier: " + mcmStr(annotation.modifier))
End Sub

Private Sub ProfileData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame

```

```

    Call DatumsStr(annotation)
End Sub
Private Sub SurfaceFinishData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Surface Finish Tolerance Compartment")
    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub
Private Sub TangencyData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Tangency Tolerance Compartment")
    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub
Private Sub DimensionToleranceData(annotation As DimXpertDimensionTolerance)
    Dim isAngleType As Boolean
    Dim annoType As Long
    Dim upper As Double, lower As Double
    Dim plus As Double, minus As Double
    Dim boolstatus As Boolean
    annoType = annotation.Type

    isAngleType = False

    strs.Add ("")
    strs.Add ("Dimension Tolerance Compartment")
    If annoType = swDimXpertDimTol_DistanceBetween Then
        Dim distancebetween As DimXpertDistanceBetweenDimTol
        Set distancebetween = annotation
        Call DistanceBetweenData(distancebetween)
    ElseIf annoType = swDimXpertDimTol_CompositeDistanceBetween Then
        Dim compdistancebetween As DimXpertCompositeDistanceBetweenDimTol
        Set compdistancebetween = annotation
        Call CompositeDistanceBetweenData(compdistancebetween)

    ElseIf annoType = swDimXpertDimTol_CounterBore Then
        Dim counterBore As IDimXpertCounterBoreDimTol
        Set counterBore = annotation
        If counterBore.ReferenceFeature Is Nothing Then
            strs.Add ("Reference Feature: NULL")
        Else
            strs.Add ("Reference Feature: " + counterBore.ReferenceFeature.Name)
        End If

    ElseIf annoType = swDimXpertDimTol_Depth Then

```

```

Dim depth As IDimXpertDepthDimTol
Set depth = annotation
If depth.ReferenceFeature Is Nothing Then
    strsr.Add ("Reference Feature: NULL")
Else
    strsr.Add ("Reference Feature: " + depth.ReferenceFeature.Name)
End If

ElseIf annoType = swDimXpertDimTol_CounterSinkDiameter Then
    Dim countersink As IDimXpertCounterSinkDiameterDimTol
    Set countersink = annotation
    If countersink.ReferenceFeature Is Nothing Then
        strsr.Add ("Reference Feature: NULL")
    Else
        strsr.Add ("Reference Feature: " + countersink.ReferenceFeature.Name)
    End If

ElseIf annoType = swDimXpertDimTol_ChamferDimension Then

    Select Case annotation.ChamferType
    Case swDimXpertChamferDimensionType_Angle
        strsr.Add ("Chamfer Dimension Type: Angle")
        isAngleType = True
    Case swDimXpertChamferDimensionType_LinearDistance1
        strsr.Add ("Chamfer Dimension Type: Distance 1")
    Case swDimXpertChamferDimensionType_LinearDistance2
        strsr.Add ("Chamfer Dimension Type: Distance 2")
    End Select

ElseIf annoType = swDimXpertDimTol_AngleBetween Then
    isAngleType = True
    Dim angleBetween As IDimXpertAngleBetweenDimTol
    Set angleBetween = annotation
    ' the origin and tolerance feature
    strsr.Add ("Origin Feature: " + angleBetween.OriginFeature.Name)

    ' is supplement angle
    strsr.Add ("Supplement Angle: " + IIf(angleBetween.Supplement, "True", "False"))

ElseIf annoType = swDimXpertDimTol_CounterSinkAngle Then
    isAngleType = True
    Dim countersinkAngle As IDimXpertCounterSinkAngleDimTol
    Set countersinkAngle = annotation

```

```

    If countersinkAngle.ReferenceFeature Is Nothing Then
        strsr.Add ("Reference Feature: NULL")
    Else
        strsr.Add ("Reference Feature: " + countersinkAngle.ReferenceFeature.Name)
    End If

ElseIf annoType = swDimXpertDimTol_ConeAngle Then
    isAngleType = True
End If

' conversion for radians to degrees when dimension type is angle
Dim dbl As Double
If isAngleType Then
    dbl = 57.2957795130823
Else
    dbl = 1#
End If

' the nominal, and upper and lower limits of size of the dimension
strsr.Add ("")
strsr.Add ("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue * dbl))
boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
strsr.Add ("Get Upper Limit: " + FormatDouble(upper * dbl))
strsr.Add ("Get Lower Limit: " + FormatDouble(lower * dbl))

' the upper and lower tolerance value by type
Select Case annotation.DimensionType

Case swDimXpertDimTolType_BlockTolerance
    strsr.Add ("Dimension Type: Block Tolerance")

' block tolerance
Case swDimXpertDimTolType_BlockToleranceNoNominal
    Dim blockTols As DimXpertBlockTolerances
    Set blockTols = dimXpertPart.GetBlockTolerances

    Select Case blockTols.Type
        Case swDimXpertBlockToleranceType_ASMEInch
            strsr.Add ("Dimension Type: Block Tolerance No Nominal")
            If isAngleType Then
                strsr.Add ("Angular Block Tolerance")
            Else
                strsr.Add ("Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
            End If
        End Select
    End Select

```

```

        End If
        Case swDimXpertBlockToleranceType_ISO2768
            strsr.Add ("Dimension Type: General Tolerance")
        End Select

Case swDimXpertDimTolType_ISOLimitsAndFits
    strsr.Add ("Dimension Type: Limits and Fits")
    ' limits and fits tolerance
Case swDimXpertDimTolType_ISOLimitsAndFitsNoNominal
    strsr.Add ("Dimension Type: Limits and Fits No Nominal")
    strsr.Add ("Limits and Fits: " + annotation.LimitsAndFitsCode)
    ' limit dimension
Case swDimXpertDimTolType_LimitDimension
    strsr.Add ("Dimension Type: Limit Dimension")
    boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
    strsr.Add ("Get Upper Limit: " + FormatDouble(upper * dbl))
    strsr.Add ("Get Lower Limit: " + FormatDouble(lower * dbl))
Case swDimXpertDimTolType_MAXTolerance
    strsr.Add ("Dimension Type: MAXTolerance")
Case swDimXpertDimTolType_MINTolerance
    strsr.Add ("Dimension Type: MINTolerance")
Case swDimXpertDimTolType_NoTolerance
    strsr.Add ("Dimension Type: NoTolerance")
Case swDimXpertDimTolType_PlusMinusDimension
    strsr.Add ("Dimension Type: Plus Minus Dimension")

    ' plus and minus dimension
Case swDimXpertDimTolType_PlusMinusNoNominal
    strsr.Add ("Dimension Type: Plus Minus No Nominal")
    boolstatus = annotation.GetPlusAndMinusTolerance(plus, minus)
    strsr.Add ("Plus Tolerance: " + FormatDouble(plus * dbl))
    strsr.Add ("Minus Tolerance: " + FormatDouble(minus * dbl))

End Select

End Sub

Private Sub DistanceBetweenData(annotation As DimXpertDistanceBetweenDimTol)
    Dim feature As DimXpertFeature
    Dim featureFosUsage As Long
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    ' the origin and tolerance feature along with their feature of size usage (min, max,
center)
    boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
    strsr.Add ("")

```

```

    strs.Add ("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    boolstatus = annotation.GetFeature(feature, featureFosUsage)
    strs.Add ("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))

    ' The direction vector
    boolstatus = annotation.GetDirectionVector(I, J, K)
    strs.Add ("")
    strs.Add ("Direction Vector: " + FormatVector(I, J, K))

```

End Sub

```

Private Sub CompositeDistanceBetweenData(annotation As
DimXpertCompositeDistanceBetweenDimTol)

```

```

    Dim feature As DimXpertFeature
    Dim featureFosUsage As Long
    Dim plus As Double, minus As Double
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    Dim blockTols As DimXpertBlockTolerances

```

```

    ' the origin and tolerance feature along with their feature of size usage (min, max,
center)

```

```

    boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
    strs.Add ("")
    strs.Add ("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    boolstatus = annotation.GetFeature(feature, featureFosUsage)
    strs.Add ("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))

```

```

    ' the pattern locating feature

```

```

    boolstatus = annotation.GetIntraFeature(feature, featureFosUsage)

```

```

    strs.Add ("Pattern Locating Feature: " + feature.Name + " @ " +
FosUsage(featureFosUsage))

```

```

    ' The direction vector

```

```

    boolstatus = annotation.GetDirectionVector(I, J, K)
    strs.Add ("")
    strs.Add ("Direction Vector: " + FormatVector(I, J, K))

```

```

    ' the upper and lower tolerance value for the pattern location by type

```

```

Select Case annotation.DimensionType

```

```

    ' plus and minus dimension

```

```

Case swDimXpertDimTolType_PlusMinusNoNominal

```

```

    strs.Add ("Pattern Locating Dimension Type: Plus Minus No Nominal")
    boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(plus, minus)
    strs.Add (" Plus Tolerance: " + FormatDouble(plus))

```

```

        strs.Add (" Minus Tolerance: " + FormatDouble(minus))
' block tolerance
Case swDimXpertDimTolType_BlockToleranceNoNominal

    Set blockTols = dimXpertPart.GetBlockTolerances

    Select Case blockTols.Type
    Case swDimXpertBlockToleranceType_ASMEInch
        strs.Add ("Pattern Locating Dimension Type: Block Tolerance No Nominal")
        strs.Add (" Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
    Case swDimXpertBlockToleranceType_IS02768
        strs.Add ("Pattern Locating Dimension Type: General Tolerance")
    End Select

End Select

' the upper and lower tolerance value for the feature to feature location by type
Select Case annotation.DimensionTypeIntraFeature
' plus and minus dimension
Case swDimXpertDimTolType_PlusMinusNoNominal
    strs.Add ("Feature Locating Dimension Type: Plus Minus No Nominal")
    boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(plus, minus)
    strs.Add (" Plus Tolerance: " + FormatDouble(plus))
    strs.Add (" Minus Tolerance: " + FormatDouble(minus))
' block tolerance
Case swDimXpertDimTolType_BlockToleranceNoNominal

    Set blockTols = dimXpertPart.GetBlockTolerances
    Select Case blockTols.Type
    Case swDimXpertBlockToleranceType_ASMEInch
        strs.Add ("Feature locating Dimension Type: Block Tolerance No Nominal")
        strs.Add (" Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlacesIntraFeature, "##,##0"))
    Case swDimXpertBlockToleranceType_IS02768
        strs.Add ("Feature locating Dimension Type: General Tolerance")
    End Select
End Select

End Sub

Private Sub DatumsStr(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Datums:")
    Call datumStr(annotation.GetPrimaryDatums(), annotation.GetPrimaryDatumModifiers(), "
Primary:")

```

```
    Call datumStr(annotation.GetSecondaryDatums(), annotation.GetSecondaryDatumModifiers(), "Secondary:")
```

```
    Call datumStr(annotation.GetTertiaryDatums(), annotation.GetTertiaryDatumModifiers(), "Tertiary:")
```

```
End Sub
```

```
Private Sub datumStr(dats As Variant, mods As Variant, datumOrder As String)
```

```
    Dim I As Long
```

```
    Dim str As String
```

```
    Dim mcm As Long
```

```
    If IsEmpty(dats) Then
```

```
        str.Add (datumOrder + ": none")
```

```
        Exit Sub
```

```
    End If
```

```
    For I = LBound(dats) To UBound(dats)
```

```
        mcm = mods(I)
```

```
        str = str + " " + dats(I).Identifier + " @ " + mcmStr(mcm)
```

```
    Next I
```

```
    If StrComp(str, "") > 0 Then
```

```
        str.Add (datumOrder + str)
```

```
    Else
```

```
        str.Add (datumOrder + " <none>")
```

```
    End If
```

```
End Sub
```

```
' returns a string containing the height of the projected tolerance zone
```

```
Private Sub FormatProjectedZone(enabled As Boolean, height As Double)
```

```
    If enabled = True Then
```

```
        str.Add (" Projected Zone: True")
```

```
        str.Add (" Zone Height: " + FormatDouble(height))
```

```
    Else
```

```
        str.Add (" Projected Zone: False")
```

```
    End If
```

```
End Sub
```

```
Private Function mcmStr(mcm As Long) As String
```

```
    Dim str
```

```
    Select Case mcm
```

```
    Case swDimXpertMCM_LMC
```

```
        str = "LMC"
```

```
    Case swDimXpertMCM_MMC
```

```
        str = "MMC"
```

```
Case swDimXpertMCM_NoMCM
```

```
    str = "NoMCM"
```

```
Case swDimXpertMCM_RFS
```

```
    str = "RFS"
```

```
End Select
```

```
mcmStr = str
```

```
End Function
```

```
' returns a string containing the type of position tolerance zone used
```

```
Private Function PositionZoneType(typ As Long) As String
```

```
    Dim str As String
```

```
    Select Case typ
```

```
    Case swDimXpertPositionZoneType_CylindricalPosition
```

```
        str = "Cylindrical"
```

```
    Case swDimXpertPositionZoneType_PlanarPosition
```

```
        str = "Planar"
```

```
    Case swDimXpertPositionZoneType_SphericalPosition
```

```
        str = "Spherical"
```

```
    Case swDimXpertPositionZoneType_Boundary
```

```
        str = "Boundary"
```

```
    Case swDimXpertPositionZoneType_RadialPositionArc
```

```
        str = "RadialPositionArc"
```

```
    Case swDimXpertPositionZoneType_RadialPositionPlanar
```

```
        str = "RadialPositionPlanar"
```

```
    Case Else
```

```
        str = "N/A"
```

```
    End Select
```

```
    PositionZoneType = str
```

```
End Function
```

```
' returns a string containing the names of the SW display entities
```

```
Private Function DisplayEntity(annotation As DimXpertAnnotation) As String
```

```
    Dim str As String
```

```
    Dim dispEnt As Object
```

```
    Dim swAnnot As SldWorks.annotation
```

```
'Set dispEnt = swDimXpert.GetDisplayEntity(annot)
```

```
Set dispEnt = annotation.GetDisplayEntity
```

```
If Not dispEnt Is Nothing Then
```

```
    If TypeOf dispEnt Is SldWorks.annotation Then
```

```
        Set swAnnot = dispEnt
```

```
        str = swAnnot.GetName
```

```

        End If
    End If

    DisplayEntity = str
End Function
' returns a string containing the feature of size usage (min, max, center)
Private Function FosUsage(value As Long) As String
    Dim str
    Select Case value
    Case swDimXpertDistanceFosUsage_Center
        str = "Center"
    Case swDimXpertDistanceFosUsage_MaximumSide
        str = "Max"
    Case swDimXpertDistanceFosUsage_MinimumSide
        str = "Min"
    Case Else
        str = "N/A"
    End Select
    FosUsage = str
End Function
Private Function FormatVector(I As Double, J As Double, K As Double) As String
    Dim str
    str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K)

    FormatVector = str
End Function
Private Function FormatDouble(value As Double) As String
    Dim str

    str = Format(value, "##,##0.0000")

    FormatDouble = str
End Function
Private Function RadiansToDegrees(value As Double) As String
    Dim str

    str = Format((value * 57.2957795130823), "##,##0.00")

    RadiansToDegrees = str
End Function
Private Function annotationTypeNameFromObject(anno As DimXpertAnnotation) As String
    annotationTypeNameFromObject = annotationTypeNameFromTypeNumber(anno.Type)
End Function
Private Function annotationTypeNameFromTypeNumber(annoTypeIndex As Long) As String
    Select Case annoTypeIndex

```

```
Case swDimXpertDimTol_DistanceBetween
    annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
Case swDimXpertDimTol_CounterBore
    annotationTypeNameFromTypeNumber = "CounterBore Dim"
Case swDimXpertDimTol_Depth
    annotationTypeNameFromTypeNumber = "Depth Dim"
Case swDimXpertDimTol_CounterSinkDiameter
    annotationTypeNameFromTypeNumber = "CounterSinkDiameter Dim"
Case swDimXpertDimTol_ChamferDimension
    annotationTypeNameFromTypeNumber = "ChamferDimension Dim"
Case swDimXpertDimTol_AngleBetween
    annotationTypeNameFromTypeNumber = "AngleBetween Dim"
Case swDimXpertDimTol_CounterSinkAngle
    annotationTypeNameFromTypeNumber = "CounterSinkAngle Dim"
Case swDimXpertDimTol_ConeAngle
    annotationTypeNameFromTypeNumber = "ConeAngle Dim"
Case swDimXpertDimTol_Diameter
    annotationTypeNameFromTypeNumber = "Diameter Dim"
Case swDimXpertDimTol_Length
    annotationTypeNameFromTypeNumber = "Length Dim"
Case swDimXpertDimTol_Radius
    annotationTypeNameFromTypeNumber = "Radius Dim"
Case swDimXpertDimTol_Width
    annotationTypeNameFromTypeNumber = "Width Dim"
Case swDimXpertDimTol_CompositeDistanceBetween
    annotationTypeNameFromTypeNumber = "CompositeDistanceBetween Dim"
Case swDimXpertDatum
    annotationTypeNameFromTypeNumber = "Datum"
Case swDimXpertGeoTol_Position
    annotationTypeNameFromTypeNumber = "Position Tol"
Case swDimXpertGeoTol_CompositePosition
    annotationTypeNameFromTypeNumber = "CompositePosition Tol"
Case swDimXpertGeoTol_Symmetry
    annotationTypeNameFromTypeNumber = "Symmetry Tol"
Case swDimXpertGeoTol_Concentricity
    annotationTypeNameFromTypeNumber = "Concentricity Tol"
Case swDimXpertGeoTol_LineProfile
    annotationTypeNameFromTypeNumber = "LineProfile Tol"
Case swDimXpertGeoTol_CompositeLineProfile
    annotationTypeNameFromTypeNumber = "CompositeLineProfile Tol"
Case swDimXpertGeoTol_SurfaceProfile
    annotationTypeNameFromTypeNumber = "SurfaceProfile Tol"
Case swDimXpertGeoTol_CompositeSurfaceProfile
    annotationTypeNameFromTypeNumber = "CompositeSurfaceProfile Tol"
```

```

Case swDimXpertGeoTol_Angularity
    annotationTypeNameFromTypeNumber = "Angularity Tol"
Case swDimXpertGeoTol_Parallelism
    annotationTypeNameFromTypeNumber = "Parallelism Tol"
Case swDimXpertGeoTol_Perpendicularity
    annotationTypeNameFromTypeNumber = "Perpendicularity Tol"
Case swDimXpertGeoTol_TotalRunout
    annotationTypeNameFromTypeNumber = "TotalRunout Tol"
Case swDimXpertGeoTol_CircularRunout
    annotationTypeNameFromTypeNumber = "CircularRunout Tol"
Case swDimXpertGeoTol_Flatness
    annotationTypeNameFromTypeNumber = "Flatness Tol"
Case swDimXpertGeoTol_Circularity
    annotationTypeNameFromTypeNumber = "Circularity Tol"
Case swDimXpertGeoTol_Cylindricity
    annotationTypeNameFromTypeNumber = "Cylindricity Tol"
Case swDimXpertGeoTol_Straightness
    annotationTypeNameFromTypeNumber = "Straightness Tol"
Case swDimXpertGeoTol_Tangency
    annotationTypeNameFromTypeNumber = "Tangency Tol"
Case Else
    annotationTypeNameFromTypeNumber = "<unknown> " & CStr(annoTypeIndex)

End Select
End Function

```

Get DimXpert Tolerance1 Example (VB.NET)

This example shows how to build a part and get attributes for the following DimXpert annotations:

- * Circularity geometric tolerance
- * Cylindricity geometric tolerance
- * Countersink angle dimension tolerance
- * Countersink diameter dimension tolerance
- * Flatness geometric tolerance
- * Surface profile geometric tolerance

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\cover_with_geometric_tolerances.sldprt.  
' 2. Click Geometric Tolerance the DimXpert toolbar.  
'   In the Geometric Tolerance Properties dialog:  
'   a. Select Cylindricity from the Symbol dropdown.  
'   b. In the second row, select Profile of Surface from the Symbol dropdown  
'      and type a tolerance of 0.5 next to it.  
'   c. Click the outer base cylinder and click again outside the part  
'      to place the annotation.  
'   d. Click OK to close the Geometric Tolerance Properties dialog.  
' 3. In the DimXpertManager tab of the Management Panel,  
'   expand all of the nodes in the tree.  
' 4. Observe the following Dimxpert annotations:  
'   Flatness1, Circularity1, Diameter1, CounterSinkAngle1,  
'   CounterSinkDiameter1, Cylindricity1, and Surface Profile1  
' 5. Ensure that the SolidWorks.Interop.swdimxpert.dll interop assembly is loaded  
'   (right-click the project in Project Explorer and click on Add Reference >  
'   NET tab).  
' 6. Ensure that the Microsoft Scripting Runtime library is loaded  
'   (right-click the project in Project Explorer and click on Add Reference >  
'   COM tab).  
'  
' Postconditions:  
' 1. Writes the output to c:\temp\dimXpertInfo.txt.  
' 2. Inspect the Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
' -----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swdimxpert  
Imports SolidWorks.Interop.swconst  
Imports System
```

```

Imports System.Diagnostics
Imports Scripting
Partial Class SolidWorksMacro
    Dim strs As New Collection
    Dim boolstatus As Boolean
    Dim dimXpertPart As DimXpertPart
    Dim i As Double
    Dim j As Double
    Dim k As Double
    Sub Main()
        Dim swModelDoc As IModelDoc2
        swModelDoc = swApp.ActiveDoc
        If swModelDoc Is Nothing Then
            Exit Sub
        End If
        Dim fso As New Scripting.FileSystemObject()
        Dim f As Scripting.File
        Dim textStr As Scripting.TextStream
        textStr = fso.CreateTextFile("c:\temp\dimXpertInfo.txt")
        f = fso.GetFile("c:\temp\dimXpertInfo.txt")
        'textStr = f.OpenAsTextStream(2, -2)
        If textStr Is Nothing Then
            Debug.Print("Error creating temp file.")
            Exit Sub
        End If
        Call log("Starting DimXpert log...", textStr)
        Call retrieve_info_text(swApp, textStr)
        textStr.Close()
    End Sub
    Private Sub log(ByVal text As String, ByVal textStr As Scripting.TextStream)
        Debug.Print(text)
        textStr.WriteLine(text)
    End Sub
    Private Sub retrieve_info_text(ByVal swapp As SldWorks, ByVal textStr As TextStream)
        Dim dimXpertMgr As SolidWorks.Interop.sldworks.DimXpertManager
        dimXpertMgr =
swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
True)
        Call log("Model: " & swapp.IActiveDoc2.GetPathName, textStr)
        Dim dimXpertPartObj As DimXpertPart
        dimXpertPartObj = dimXpertMgr.DimXpertPart
        dimXpertPart = dimXpertPartObj
        Dim vAnnotations As Object
        vAnnotations = dimXpertPart.GetAnnotations()
        Call log("-----", textStr)
    End Sub
End Class

```

```

Call log("Annotations...", textStr)
Call log("-----", textStr)
Dim annotationTemp As DimXpertAnnotation
Dim annotationIndex As Long
For annotationIndex = 0 To UBound(vAnnotations)
    annotationTemp = vAnnotations(annotationIndex)
    Dim AnnotationDataText As Collection
    AnnotationDataText = AnnotationData(annotationTemp)
    Dim annotationTextIndex As Long
    For annotationTextIndex = 1 To AnnotationDataText.Count
        Call log(AnnotationDataText(annotationTextIndex), textStr)
    Next
Next
End Sub

Public Function AnnotationData(ByVal annotation As DimXpertAnnotation) As Collection
    Dim annoType As Long
    'general info
    Call GeneralInfo(annotation)
    annoType = annotation.Type
    If annoType = swDimXpertAnnotationType_e.swDimXpertDatum Then
        Call DatumData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Position Then
        Call PositionData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition Then
        Call CompositePositionData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry Then
        Call SymmetryData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity Then
        Call ConcentricityData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile Then
        Call LineProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile
Then
        Call CompositeLineProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile Then
        Call SurfaceProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile
Then
        Call CompositeSurfaceProfileData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity Or annoType =
swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism Or annoType =
swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity Then
        Call OrientationData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout Then
        Call TotalRunoutData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout Then

```

```

        Call CircularRunoutData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness Then
        Call FlatnessData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity Then
        Call CircularityData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity Then
        Call CylindricityData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness Then
        Call StraightnessData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency Then
        Call TangencyData(annotation)
    Else ' any of the dimension tolerance types
        Call DimensionToleranceData(annotation)
    End If
    AnnotationData = strs
End Function
Private Sub Clear(ByVal strs As Collection)
    strs.Remove(strs.Count)
    If Not strs.Count = 0 Then
        Call Clear(strs)
    End If
End Sub
Private Sub GeneralInfo(ByVal annotation As DimXpertAnnotation)
    Dim annoType As String
    Dim modelObj As Feature
    If Not strs.Count = 0 Then
        Call Clear(strs)
    End If
    strs.Add("")
    strs.Add("Name: " + annotation.Name)
    annoType = annotationTypeNameFromObject(annotation)
    strs.Add("Type: " + annoType)
    strs.Add("Display Entity: " + DisplayEntity(annotation))
    modelObj = annotation.GetModelFeature()
    If Not (modelObj Is Nothing) Then
        strs.Add("ModelFeature: " + modelObj.Name + " (" + modelObj.GetTypeName2() + ")")
    End If
End Sub
Private Sub DatumData(ByVal annotation As DimXpertDatum)
    ' the datum letter
    strs.Add("")
    strs.Add("Datum Letter: " + annotation.Identifier)
End Sub
Private Sub PositionData(ByVal annotation As DimXpertPositionTolerance)
    Dim I As Double, J As Double, K As Double

```

```

Dim enabled As Boolean, value As Double
Dim boolstatus As Boolean
strs.Add("")
strs.Add("Position Tolerance Compartment:")
' the shape of the tolerance zone
strs.Add(" Zone Type: " + PositionZoneType(annotation.ZoneType))
' the zone vector if the tolerance zone is planar
If annotation.ZoneType =
swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition Then
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add(" Direction: " + FormatVector(I, J, K))
End If
' the tolerance value
strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
' the material condition modifier applied to feature
strs.Add(" Modifier: " + mcmStr(annotation.Modifier))
' the projected tolerance zone when specified
boolstatus = annotation.GetProjectedZone(enabled, value)
Call FormatProjectedZone(enabled, value)
' the datum reference frame
Call DatumsStr(annotation)
End Sub

Private Sub CompositePositionData(ByVal annotation As DimXpertCompositePositionTolerance)
Dim I As Double, J As Double, K As Double
Dim enabled As Boolean, value As Double
Dim boolstatus As Boolean
strs.Add("")
strs.Add("Composite Position Tolerance Compartment")
' the shape of the tolerance zone
strs.Add(" Zone Type: " + PositionZoneType(annotation.ZoneType))
' the zone vector when the zone is planar
If annotation.ZoneType =
swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition Then
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add(" Direction: " + FormatVector(I, J, K))
End If
' the tolerance value
strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
' the material condition modifier
strs.Add(" Modifier: " + mcmStr(annotation.Modifier))
' the projected tolerance zone when specified
boolstatus = annotation.GetProjectedZone(enabled, value)
Call FormatProjectedZone(enabled, value)
' the datum reference frame for the pattern location
Call DatumsStr(annotation)

```

```

        ' the datum reference frame for the feature to feature location
        str.Add("Composite datums:")
        str.Add(" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
        str.Add(" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
        str.Add(" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
    End Sub

Private Sub SymmetryData(ByVal annotation As DimXpertSymmetryTolerance)
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    str.Add("")
    str.Add("Symmetry Tolerance Compartment")
    ' the tolerance value
    str.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier applied to feature
    str.Add(" Modifier: " + mcmStr(annotation.Modifier))
    ' the datum reference frame
    str.Add("")
    Call DatumsStr(annotation)
    ' the direction of the planar zone
    str.Add("")
    boolstatus = annotation.GetZoneDirection(I, J, K)
    str.Add("Planar Zone Direction: " + FormatVector(I, J, K))
End Sub

Private Sub ConcentricityData(ByVal annotation As DimXpertConcentricityTolerance)
    str.Add("")
    str.Add("Concentricity Tolerance Compartment")
    ' the tolerance value
    str.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier applied to feature
    str.Add(" Modifier: " + mcmStr(annotation.Modifier))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub TotalRunoutData(ByVal annotation As DimXpertTolerance)
    str.Add("")
    str.Add("Total Runout Tolerance Compartment")
    ' the tolerance value
    str.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub CircularRunoutData(ByVal annotation As DimXpertTolerance)
    str.Add("")

```

```

    strs.Add("Circular Runout Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub LineProfileData(ByVal annotation As DimXpertLineProfileTolerance)
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    strs.Add("")
    strs.Add("Line Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the outer (outside material) tolerance value
    strs.Add("  Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
    ' the vector normal to the profile zones
    strs.Add("")
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add("Planar Zone Vector: " + FormatVector(I, J, K))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub CompositeLineProfileData(ByVal annotation As
DimXpertCompositeLineProfileTolerance)
    strs.Add("")
    strs.Add("Composite Line Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the outer (outside material) tolerance value
    strs.Add("  Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
    ' the vector normal to the profile zones
    strs.Add("")
    boolstatus = annotation.GetPlanarZoneVector(i, j, k)
    strs.Add("Planar Zone Vector: " + FormatVector(i, j, k))
    ' the datum reference frame
    Call DatumsStr(annotation)
    ' the datum reference frame for the orientation and form
    strs.Add("Composite Datums:")
    strs.Add("  Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    strs.Add("  Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
    strs.Add("  Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
End Sub

Private Sub SurfaceProfileData(ByVal annotation As DimXpertSurfaceProfileTolerance)
    strs.Add("")

```

```

    strs.Add("Surface Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the outer (outside material) tolerance value
    strs.Add("  Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub CompositeSurfaceProfileData(ByVal annotation As
DimXpertCompositeSurfaceProfileTolerance)
    strs.Add("")
    strs.Add("Composite Surface Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance Upper Tier: " + FormatDouble(annotation.Tolerance))
    ' the outer tolerance value
    strs.Add("  Outer Tolerance Upper Tier: " +
FormatDouble(annotation.OuterToleranceValue))
    ' the datum reference frame for the location
    Call DatumsStr(annotation)
    ' the datum reference frame for the orientation and form
    strs.Add("Composite Datums:")
    strs.Add("  Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    strs.Add("  Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
    strs.Add("  Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
End Sub

Private Sub OrientationData(ByVal annotation As DimXpertOrientationTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim annoType As Long
    annoType = annotation.Type
    ' the type or orientation tolerance
    If annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity Then
        strs.Add("Orientation Type:  Perpendicularity")
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism Then
        strs.Add("Orientation Type:  Parallelism")
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity Then
        strs.Add("Orientation Type:  Angularity")
    End If
    strs.Add("")
    strs.Add("Orientation Tolerance Compartment:")
    ' the shape of the tolerance zone
    Select Case annotation.ZoneType
        Case swDimXpertOrientationZoneType_e.swDimXpertOrientationZoneType_Cylindrical
            strs.Add("  Zone Type: Cylindrical")
    End Select
End Sub

```

```

        Case swDimXpertOrientationZoneType_e.swDimXpertOrientationZoneType_Planar
            strsr.Add(" Zone Type: Planar")
            boolstatus = annotation.GetPlanarZoneVector(I, J, K)
            strsr.Add("Planar Zone Vector: " + FormatVector(I, J, K))
        End Select
        ' the tolerance value
        strsr.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
        'material condition modifier applied to feature
        strsr.Add(" Modifier: " + mcmStr(annotation.Modifier))
        ' the projected tolerance zone when specified
        boolstatus = annotation.GetProjectedZone(enabled, value)
        Call FormatProjectedZone(enabled, value)
        ' is tangent plane
        strsr.Add(" IsTangentPlane: " + IIf(annotation.IsTangentPlane, "True", "False"))
        ' the datum reference frame
        Call DatumsStr(annotation)
    End Sub

Private Sub FlatnessData(ByVal annotation As DimXpertFlatnessTolerance)
    Dim enabled As Boolean
    Dim length As Double, width As Double
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    strsr.Add("")
    strsr.Add("Flatness Tolerance Compartment:")
    ' the tolerance value
    strsr.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the per unit area data
    boolstatus = annotation.GetPerUnitArea(enabled, length, width, I, J, K)
    strsr.Add(" Per Unit Area: " + IIf(enabled, "True", "False"))
    If enabled Then
        strsr.Add(" Per Unit Length: " + FormatDouble(length))
        strsr.Add(" Per Unit Width: " + FormatDouble(width))
        strsr.Add(" Per Unit Direction: " + FormatVector(I, J, K))
    End If
End Sub

Private Sub CircularityData(ByVal annotation As DimXpertTolerance)
    strsr.Add("")
    strsr.Add("Circularity Tolerance Compartment")
    ' the tolerance value
    strsr.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub CylindricityData(ByVal annotation As DimXpertTolerance)
    strsr.Add("")
    strsr.Add("Cylindricity Tolerance Compartment")
    ' the tolerance value

```

```

        str.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub
Private Sub StraightnessData(ByVal annotation As DimXpertStraightnessTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean
    Dim dist As Double
    Dim boolstatus As Boolean
    'tolerance compartment info
    str.Add("")
    str.Add("Straightness Tolerance Compartment")
    'type or shape of the tolerance zone
    Select Case annotation.ZoneType
        Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_Cylindrical
            str.Add(" Zone Type: Cylindrical")
        Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_PlanarMedian
            str.Add(" Zone Type: Planar Median")
        Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_Surface
            str.Add(" Zone Type: Surface Straightness")
            boolstatus = annotation.GetPlanarZoneVector(I, J, K)
            str.Add(" Zone Vector: " + FormatVector(I, J, K))
    End Select
    ' the tolerance value
    str.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    'per unit length
    boolstatus = annotation.GetPerUnitLength(enabled, dist)
    str.Add(" Per Unit Length: " + IIf(enabled, "True", "False"))
    If enabled Then
        str.Add(" Per Unit Length Distance: " + FormatDouble(dist))
    End If
    ' the material condition modifier
    str.Add(" Modifier: " + mcmStr(annotation.Modifier))
End Sub
Private Sub ProfileData(ByVal annotation As DimXpertTolerance)
    str.Add("")
    str.Add("Profile Tolerance Compartment")
    ' the tolerance value
    str.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub
Private Sub SurfaceFinishData(ByVal annotation As DimXpertTolerance)
    str.Add("")
    str.Add("Surface Finish Tolerance Compartment")
    ' the tolerance value
    str.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))

```

```

End Sub
Private Sub TangencyData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Tangency Tolerance Compartment")
    ' the tolerance value
    strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub
Private Sub DimensionToleranceData(ByVal annotation As DimXpertDimensionTolerance)
    Dim isAngleType As Boolean
    Dim annoType As Long
    Dim upper As Double, lower As Double
    Dim plus As Double, minus As Double
    Dim boolstatus As Boolean
    annoType = annotation.Type
    isAngleType = False
    strs.Add("")
    strs.Add("Dimension Tolerance Compartment")
    If annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween Then
        Call DistanceBetweenData(annotation)
    ElseIf annoType =
swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween Then
        Call CompositeDistanceBetweenData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore Then
        If annotation.ReferenceFeature Is Nothing Then
            strs.Add("Reference Feature: NULL")
        Else
            strs.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
        End If
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_Depth Then
        If annotation.ReferenceFeature Is Nothing Then
            strs.Add("Reference Feature: NULL")
        Else
            strs.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
        End If
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter
Then
        Dim csDiameter As IDimXpertCounterSinkDiameterDimTol
        csDiameter = annotation
        If csDiameter.ReferenceFeature Is Nothing Then
            strs.Add("Reference Feature: NULL")
        Else
            strs.Add("Reference Feature: " + csDiameter.ReferenceFeature.Name)
        End If
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension Then
        Dim chamferAnno As IDimXpertChamferDimTol

```

```

    chamferAnno = annotation
    Select Case chamferAnno.ChamferType
        Case swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_Angle
            strs.Add("Chamfer Dimension Type: Angle")
            isAngleType = True
        Case
            swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_LinearDistance1
                strs.Add("Chamfer Dimension Type: Distance 1")
            Case
                swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_LinearDistance2
                    strs.Add("Chamfer Dimension Type: Distance 2")
    End Select
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween Then
        isAngleType = True
        ' the origin and tolerance feature
        strs.Add("Origin Feature: " + annotation.OriginFeature.Name)
        ' is supplement angle
        strs.Add("Supplement Angle: " + IIf(annotation.Supplement, "True", "False"))
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle Then
        Dim csAngle As IDimXpertCounterSinkAngleDimTol
        csAngle = annotation
        isAngleType = True
        If csAngle.ReferenceFeature Is Nothing Then
            strs.Add("Reference Feature: NULL")
        Else
            strs.Add("Reference Feature: " + csAngle.ReferenceFeature.Name)
        End If
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle Then
        isAngleType = True
    End If
    ' conversion for radians to degrees when dimension type is angle
    Dim dbl As Double
    If isAngleType Then
        dbl = 57.2957795130823
    Else
        dbl = 1.0#
    End If
    ' the nominal, and upper and lower limits of size of the dimension
    strs.Add("")
    strs.Add("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue *
dbl))
    boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
    strs.Add("Get Upper Limit: " + FormatDouble(upper * dbl))
    strs.Add("Get Lower Limit: " + FormatDouble(lower * dbl))
    ' the upper and lower tolerance value by type
    Select Case annotation.DimensionType

```

```

        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockTolerance
            strs.Add("Dimension Type: Block Tolerance")
            ' block tolerance
    Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
        Dim blockTols As DimXpertBlockTolerances
        blockTols = dimXpertPart.GetBlockTolerances
        Select Case blockTols.Type
            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                strs.Add("Dimension Type: Block Tolerance No Nominal")
                If isAngleType Then
                    strs.Add("Angular Block Tolerance")
                Else
                    strs.Add("Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
                End If
            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
                strs.Add("Dimension Type: General Tolerance")
        End Select
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFits
        strs.Add("Dimension Type: Limits and Fits")
        ' limits and fits tolerance
    Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFitsNoNominal
        strs.Add("Dimension Type: Limits and Fits No Nominal")
        strs.Add("Limits and Fits: " + annotation.LimitsAndFitsCode)
        ' limit dimension
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_LimitDimension
        strs.Add("Dimension Type: Limit Dimension")
        boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
        strs.Add("Get Upper Limit: " + FormatDouble(upper * dbl))
        strs.Add("Get Lower Limit: " + FormatDouble(lower * dbl))
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MAXTolerance
        strs.Add("Dimension Type: MAXTolerance")
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MINTolerance
        strs.Add("Dimension Type: MINTolerance")
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_NoTolerance
        strs.Add("Dimension Type: NoTolerance")
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusDimension
        strs.Add("Dimension Type: Plus Minus Dimension")
        ' plus and minus dimension
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
        strs.Add("Dimension Type: Plus Minus No Nominal")
        boolstatus = annotation.GetPlusAndMinusTolerance(plus, minus)
        strs.Add("Plus Tolerance: " + FormatDouble(plus * dbl))
        strs.Add("Minus Tolerance: " + FormatDouble(minus * dbl))

```

```

        End Select
    End Sub

    Private Sub DistanceBetweenData(ByVal annotation As DimXpertDistanceBetweenDimTol)
        Dim feature As DimXpertFeature
        Dim featureFosUsage As Long
        Dim I As Double, J As Double, K As Double
        Dim boolstatus As Boolean
        feature = Nothing
        ' the origin and tolerance feature along with their feature of size usage (min, max,
center)
        boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
        str.Add("")
        str.Add("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
        boolstatus = annotation.GetFeature(feature, featureFosUsage)
        str.Add("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
        ' The direction vector
        boolstatus = annotation.GetDirectionVector(I, J, K)
        str.Add("")
        str.Add("Direction Vector: " + FormatVector(I, J, K))
    End Sub

    Private Sub CompositeDistanceBetweenData(ByVal annotation As
DimXpertCompositeDistanceBetweenDimTol)
        Dim feature As DimXpertFeature
        Dim featureFosUsage As Long
        Dim plus As Double, minus As Double
        Dim I As Double, J As Double, K As Double
        Dim boolstatus As Boolean
        feature = Nothing
        ' the origin and tolerance feature along with their feature of size usage (min, max,
center)
        boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
        str.Add("")
        str.Add("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
        boolstatus = annotation.GetFeature(feature, featureFosUsage)
        str.Add("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
        ' the pattern locating feature
        boolstatus = annotation.GetIntraFeature(feature, featureFosUsage)
        str.Add("Pattern Locating Feature: " + feature.Name + " @ " +
FosUsage(featureFosUsage))
        ' The direction vector
        boolstatus = annotation.GetDirectionVector(I, J, K)
        str.Add("")
        str.Add("Direction Vector: " + FormatVector(I, J, K))
        ' the upper and lower tolerance value for the pattern location by type
        Dim dimTol As IDimXpertDimensionTolerance
        dimTol = annotation

```

```

Select Case dimTol.DimensionType
    ' plus and minus dimension
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
        dimTol.GetPlusAndMinusTolerance(plus, minus)
        strsr.Add("Pattern Locating Dimension Type: Plus Minus No Nominal")
        strsr.Add(" Plus Tolerance: " + FormatDouble(plus))
        strsr.Add(" Minus Tolerance: " + FormatDouble(minus))
    ' block tolerance
    Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
        Dim blockTols As DimXpertBlockTolerances
        blockTols = dimXpertPart.GetBlockTolerances
        Select Case blockTols.Type
            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                strsr.Add("Pattern Locating Dimension Type: Block Tolerance No
Nominal")
                strsr.Add(" Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
                strsr.Add("Pattern Locating Dimension Type: General Tolerance")
        End Select
    End Select
End Select
' the upper and lower tolerance value for the feature to feature location by type
Select Case annotation.DimensionTypeIntraFeature
    ' plus and minus dimension
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
        strsr.Add("Feature Locating Dimension Type: Plus Minus No Nominal")
        boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(plus, minus)
        strsr.Add(" Plus Tolerance: " + FormatDouble(plus))
        strsr.Add(" Minus Tolerance: " + FormatDouble(minus))
    ' block tolerance
    Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
        Dim blockTols As DimXpertBlockTolerances
        blockTols = dimXpertPart.GetBlockTolerances
        Select Case blockTols.Type
            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                strsr.Add("Feature locating Dimension Type: Block Tolerance No
Nominal")
                strsr.Add(" Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlacesIntraFeature, "##,##0"))
            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
                strsr.Add("Feature locating Dimension Type: General Tolerance")
        End Select
    End Select
End Select
End Sub
Private Sub DatumsStr(ByVal annotation As DimXpertTolerance)

```

```

        strs.Add("")
        strs.Add("Datums:")
        Call datumStr(annotation.GetPrimaryDatums(), annotation.GetPrimaryDatumModifiers(), "
Primary:")
        Call datumStr(annotation.GetSecondaryDatums(),
annotation.GetSecondaryDatumModifiers(), "  Secondary:")
        Call datumStr(annotation.GetTertiaryDatums(), annotation.GetTertiaryDatumModifiers(),
"    Tertiary:")
    End Sub

    Private Sub datumStr(ByVal dats As Object, ByVal mods As Object, ByVal datumOrder As
String)
        Dim I As Long
        Dim str As String
        Dim mcm As Long
        If IsNothing(dats) Then
            Exit Sub
        End If
        str = ""
        For I = LBound(dats) To UBound(dats)
            mcm = mods(I)
            str = str + "  " + dats(I).Identifier + " @ " + mcmStr(mcm)
        Next I
        If StrComp(str, "") > 0 Then
            strs.Add(datumOrder + str)
        Else
            strs.Add(datumOrder + " <none>")
        End If
    End Sub

    ' returns a string containing the heith of the projected tolerance zone
    Private Sub FormatProjectedZone(ByVal enabled As Boolean, ByVal height As Double)
        If enabled = True Then
            strs.Add("  Projected Zone:  True")
            strs.Add("  Zone Height: " + FormatDouble(height))
        Else
            strs.Add("  Projected Zone:  False")
        End If
    End Sub

    Private Function mcmStr(ByVal mcm As Long) As String
        Dim str As String
        str = ""
        Select Case mcm
            Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_LMC
                str = "LMC"
            Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_MMC
                str = "MMC"
            Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_NoMCM

```

```

        str = "NoMCM"
    Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_RFS
        str = "RFS"
    End Select
    mcmStr = str
End Function

' returns a string containing the type of position tolerance zone used
Private Function PositionZoneType(ByVal typ As Long) As String
    Dim str As String
    Select Case typ
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_CylindricalPosition
            str = "Cylindrical"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition
            str = "Planar"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_SphericalPosition
            str = "Spherical"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_Boundary
            str = "Boundary"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_RadialPositionArc
            str = "RadialPositionArc"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_RadialPositionPlanar
            str = "RadialPositionPlanar"
        Case Else
            str = "N/A"
    End Select
    PositionZoneType = str
End Function

' returns a string containing the names of the SW display entities
Private Function DisplayEntity(ByVal annotation As DimXpertAnnotation) As String
    Dim str As String
    Dim dispEnt As Object
    Dim swAnnot As SolidWorks.Interop.sldworks.Annotation
    str = ""
    'Set dispEnt = swDimXpert.GetDisplayEntity(annot)
    dispEnt = annotation.GetDisplayEntity()
    If Not dispEnt Is Nothing Then
        If TypeOf dispEnt Is SolidWorks.Interop.sldworks.Annotation Then
            swAnnot = dispEnt
            str = swAnnot.GetName
        End If
    End If
    DisplayEntity = str
End Function

' returns a string containing the feature of size usage (min, max, center)
Private Function FosUsage(ByVal value As Long) As String

```

```

Dim str As String
Select Case value
    Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_Center
        str = "Center"
    Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MaximumSide
        str = "Max"
    Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MinimumSide
        str = "Min"
    Case Else
        str = "N/A"
End Select
FosUsage = str
End Function

Private Function FormatVector(ByVal I As Double, ByVal J As Double, ByVal K As Double) As String
    Dim str As String
    str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K)
    FormatVector = str
End Function

Private Function FormatDouble(ByVal value As Double) As String
    Dim str As String
    str = Format(value, "##,##0.0000")
    FormatDouble = str
End Function

Private Function RadiansToDegrees(ByVal value As Double) As String
    Dim str As String
    str = Format((value * 57.2957795130823), "##,##0.00")
    RadiansToDegrees = str
End Function

Private Function annotationTypeNameFromObject(ByVal anno As DimXpertAnnotation) As String
    annotationTypeNameFromObject = annotationTypeNameFromTypeNumber(anno.Type)
End Function

Private Function annotationTypeNameFromTypeNumber(ByVal annoTypeIndex As Long) As String
    Select Case annoTypeIndex
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween
            annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore
            annotationTypeNameFromTypeNumber = "CounterBore Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_Depth
            annotationTypeNameFromTypeNumber = "Depth Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter
            annotationTypeNameFromTypeNumber = "CounterSinkDiameter Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension
            annotationTypeNameFromTypeNumber = "ChamferDimension Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween

```

```
        annotationTypeNameFromTypeNumber = "AngleBetween Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle
        annotationTypeNameFromTypeNumber = "CounterSinkAngle Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle
        annotationTypeNameFromTypeNumber = "ConeAngle Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_Diameter
        annotationTypeNameFromTypeNumber = "Diameter Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_Length
        annotationTypeNameFromTypeNumber = "Length Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_Radius
        annotationTypeNameFromTypeNumber = "Radius Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_Width
        annotationTypeNameFromTypeNumber = "Width Dim"
Case swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween
        annotationTypeNameFromTypeNumber = "CompositeDistanceBetween Dim"
Case swDimXpertAnnotationType_e.swDimXpertDatum
        annotationTypeNameFromTypeNumber = "Datum"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Position
        annotationTypeNameFromTypeNumber = "Position Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition
        annotationTypeNameFromTypeNumber = "CompositePosition Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry
        annotationTypeNameFromTypeNumber = "Symmetry Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity
        annotationTypeNameFromTypeNumber = "Concentricity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile
        annotationTypeNameFromTypeNumber = "LineProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile
        annotationTypeNameFromTypeNumber = "CompositeLineProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile
        annotationTypeNameFromTypeNumber = "SurfaceProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile
        annotationTypeNameFromTypeNumber = "CompositeSurfaceProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity
        annotationTypeNameFromTypeNumber = "Angularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism
        annotationTypeNameFromTypeNumber = "Parallelism Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity
        annotationTypeNameFromTypeNumber = "Perpendicularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout
        annotationTypeNameFromTypeNumber = "TotalRunout Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout
        annotationTypeNameFromTypeNumber = "CircularRunout Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness
        annotationTypeNameFromTypeNumber = "Flatness Tol"
```

```

    Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity
        annotationTypeNameFromTypeNumber = "Circularity Tol"
    Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity
        annotationTypeNameFromTypeNumber = "Cylindricity Tol"
    Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness
        annotationTypeNameFromTypeNumber = "Straightness Tol"
    Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency
        annotationTypeNameFromTypeNumber = "Tangency Tol"
    Case Else
        annotationTypeNameFromTypeNumber = "<unknown> " & CStr(annoTypeIndex)
    End Select
End Function
Public swApp As SldWorks
End Class

```

Get DimXpert Tolerance2 Example (VBA)

This example shows how to build a part and get attributes for the following DimXpert annotations:

- * Distance-between dimension tolerance
- * Composite distance-between dimension tolerance
- * Symmetry geometric tolerance
- * Radius dimension tolerance
- * Diameter dimension tolerance

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\block.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars  
'  
' Create a distance-between dimension tolerance:  
' 1. Click Location Dimension on the DimXpert toolbar.  
' 2. Click the left front face of the block.  
' 3. Select Create Compound Plane from the DimXpert shortcut toolbar.  
' 4. Click the right front face of the block.  
' 5. Click the green check mark to accept the faces.  
' 6. Click the back face of the block to complete the location dimension.  
' 7. Click to position the location dimension.  
' 8. Press Esc to end dimensioning.  
' 9. Observe the following Dimxpert annotations: DistanceBetween1  
  
' Create composite distance-between and radius dimension tolerances:  
' 1. Click Auto Dimension Scheme on the DimXpert toolbar.  
' 2. Select Part type of Prismatic.  
' 3. Select Tolerance type of Plus and Minus.  
' 4. For Primary Datum, select the face in front of right hole.  
' 5. For Secondary Datum, select the top face of block.  
' 6. Select all feature filters.  
' 7. Click the green check mark to accept the DimXpert settings.  
' 8. Observe the following Dimxpert annotations:  
'   Diameter1, DistanceBetween5, and Radius1.  
  
' Create a symmetry geometric tolerance:  
' 1. Click Datum on the DimXpert toolbar.  
' 2. Click the face in front of the left hole and click  
'   to place Datum A.  
' 3. Click the green check mark to finish the Datum definition.  
' 4. Click Geometric Tolerance on the DimXpert toolbar.
```

```
' In the Geometric Tolerance Properties dialog:
' a. Select Symmetry from the Symbol dropdown.
' b. Type A for Primary Datum.
' c. Click the face in front of the right hole.
' d. Click to place the annotation.
' e. Click OK to close the Geometric Tolerance Properties dialog.
' 5. Observe the following Dimxpert annotations: Symmetry1.
```

```
' Before running this macro:
```

```
' 1. Open an Immediate window.
' 2. Ensure that the SOLIDWORKS DimXpert type library is loaded
'    in Tools > References.
' 3. Ensure that the Microsoft Scripting Runtime library is loaded
'    in Tools > References.
```

```
' Postconditions:
```

```
' 1. Writes the output to c:\temp\dimXpertInfo.txt.
' 2. Inspect the Immediate Window.
```

```
' NOTE: Because the part is used elsewhere, do not save changes.
```

```
Option Explicit
```

```
Dim strs As New Collection
```

```
Dim dimXpertPart As SwDimXpert.dimXpertPart
```

```
Dim patternTreatment As swDimXpertPatternTreatmentType_e
```

```
Sub Main()
```

```
    Dim swapp As SldWorks.SldWorks
```

```
    Set swapp = Application.SldWorks
```

```
Dim swModelDoc As SldWorks.ModelDoc2
```

```
Set swModelDoc = swapp.ActiveDoc
```

```
If swModelDoc Is Nothing Then
```

```
    Exit Sub
```

```
End If
```

```
Dim f As New FileSystemObject
```

```
Dim textStr As TextStream
```

```
Set textStr = f.CreateTextFile("C:\temp\dimXpertInfo.txt", True)
```

```
If textStr Is Nothing Then
```

```
    Debug.Print "Error creating temp file."
```

```
    Exit Sub
```

```
End If
```

```
Call log("Starting DimXpert log...", textStr)
```

```
Call retrieve_info_text(swapp, textStr)
```

```
textStr.Close
```

```

End Sub

Private Sub log(text As String, textStr As TextStream)
    Debug.Print text
    textStr.WriteLine (text)
End Sub

Private Sub retrieve_info_text(swapp As SldWorks.SldWorks, textStr As TextStream)
    Dim dimXpertMgr As SldWorks.DimXpertManager

    Set dimXpertMgr =
swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
True)

    Call log("Model: " & swapp.IActiveDoc2.GetPathName, textStr)
    Dim dimXpertPartObj As dimXpertPart
    Set dimXpertPartObj = dimXpertMgr.dimXpertPart

    Set dimXpertPart = dimXpertPartObj

    Dim vAnnotations As Variant

    vAnnotations = dimXpertPart.GetAnnotations()

    Call log("-----", textStr)
    Call log("Annotations...", textStr)
    Call log("-----", textStr)

    Dim annotationTemp As DimXpertAnnotation
    Dim annotationIndex As Long
    For annotationIndex = 0 To UBound(vAnnotations)
        Set annotationTemp = vAnnotations(annotationIndex)

        Dim AnnotationDataText As Collection

        Set AnnotationDataText = AnnotationData(annotationTemp)
        Dim annotationTextIndex As Long
        For annotationTextIndex = 1 To AnnotationDataText.Count
            Call log(AnnotationDataText(annotationTextIndex), textStr)
        Next
    Next

Next

End Sub

Public Function AnnotationData(annotation As DimXpertAnnotation) As Collection
    Dim annoType As Long

```

```

'general info
Call GeneralInfo(annotation)
annoType = annotation.Type
If annoType = swDimXpertDatum Then
    Call DatumData(annotation)

ElseIf annoType = swDimXpertGeoTol_Position Then
    Call PositionData(annotation)

ElseIf annoType = swDimXpertGeoTol_CompositePosition Then
    Call CompositePositionData(annotation)

ElseIf annoType = swDimXpertGeoTol_Symmetry Then
    Call SymmetryData(annotation)

ElseIf annoType = swDimXpertGeoTol_Concentricity Then
    Call ConcentricityData(annotation)

ElseIf annoType = swDimXpertGeoTol_LineProfile Then
    Call LineProfileData(annotation)

ElseIf annoType = swDimXpertGeoTol_CompositeLineProfile Then
    Call CompositeLineProfileData(annotation)

ElseIf annoType = swDimXpertGeoTol_SurfaceProfile Then
    Call SurfaceProfileData(annotation)

ElseIf annoType = swDimXpertGeoTol_CompositeSurfaceProfile Then
    Call CompositeSurfaceProfileData(annotation)

ElseIf annoType = swDimXpertGeoTol_Angularity Or annoType = swDimXpertGeoTol_Parallelism
Or annoType = swDimXpertGeoTol_Perpendicularity Then
    Call OrientationData(annotation)

ElseIf annoType = swDimXpertGeoTol_TotalRunout Then
    Call TotalRunoutData(annotation)

ElseIf annoType = swDimXpertGeoTol_CircularRunout Then
    Call CircularRunoutData(annotation)

ElseIf annoType = swDimXpertGeoTol_Flatness Then
    Call FlatnessData(annotation)

ElseIf annoType = swDimXpertGeoTol_Circularity Then

```

```

        Call CircularityData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Cylindricity Then
        Call CylindricityData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Straightness Then
        Call StraightnessData(annotation)

    ElseIf annoType = swDimXpertGeoTol_Tangency Then
        Call TangencyData(annotation)
    Else ' any of the dimension tolerance types
        Call DimensionToleranceData(annotation)

    End If

    Set AnnotationData = strs
End Function
Private Sub Clear(strs As Collection)
    Dim n As Long
    strs.Remove (strs.Count)
    If Not strs.Count = 0 Then
        Call Clear(strs)
    End If
End Sub
Private Sub GeneralInfo(annotation As DimXpertAnnotation)
    Dim annoType As String
    Dim modelObj As Object
    Dim modelFeature As SldWorks.feature
    If Not strs.Count = 0 Then
        Call Clear(strs)
    End If

    strs.Add ("")
    strs.Add ("Name: " + annotation.Name)

    annoType = annotationTypeNameFromObject(annotation)
    strs.Add ("Type: " + annoType)
    strs.Add ("Display Entity: " + DisplayEntity(annotation))

    Set modelObj = annotation.GetModelFeature
    Set modelFeature = modelObj
    If Not (modelFeature Is Nothing) Then
        strs.Add ("ModelFeature: " + modelFeature.Name + " (" + modelFeature.GetTypeName2() +
        ")")
    End If

```

```

    End If
End Sub
Private Sub DatumData(annotation As DimXpertDatum)
    ' the datum letter
    strs.Add ("")
    strs.Add ("Datum Letter: " + annotation.Identifier)

End Sub
Private Sub PositionData(annotation As DimXpertPositionTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean

    strs.Add ("")
    strs.Add ("Position Tolerance Compartment:")

    ' the shape of the tolerance zone
    strs.Add (" Zone Type: " + PositionZoneType(annotation.zoneType))

    ' the zone vector if the tolerance zone is planar
    If annotation.zoneType = swDimXpertPositionZoneType_PlanarPosition Then
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        strs.Add (" Direction: " + FormatVector(I, J, K))
    End If

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the material condition modifier applied to feature
    strs.Add (" Modifier: " + mcmStr(annotation.modifier))

    ' the projected tolerance zone when specified
    boolstatus = annotation.GetProjectedZone(enabled, value)
    Call FormatProjectedZone(enabled, value)

    ' the datum reference frame
    Call DatumsStr(annotation)

End Sub
Private Sub CompositePositionData(annotation As DimXpertCompositePositionTolerance)

    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean

```

```

strs.Add ("")
strs.Add ("Composite Position Tolerance Compartment")

' the shape of the tolerance zone
strs.Add (" Zone Type: " + PositionZoneType(annotation.zoneType))

' the zone vector when the zone is planar
If annotation.zoneType = swDimXpertPositionZoneType_PlanarPosition Then
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add (" Direction: " + FormatVector(I, J, K))
End If

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

' the material condition modifier
strs.Add (" Modifier: " + mcmStr(annotation.modifier))

' the projected tolerance zone when specified
boolstatus = annotation.GetProjectedZone(enabled, value)
Call FormatProjectedZone(enabled, value)

' the datum reference frame for the pattern location
Call DatumsStr(annotation)

' the datum reference frame for the feature to feature location
strs.Add ("Composite datums:")
strs.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
strs.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))
strs.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))

End Sub

Private Sub SymmetryData(annotation As DimXpertSymmetryTolerance)
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    strs.Add ("")
    strs.Add ("Symmetry Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the material condition modifier applied to feature
    strs.Add (" Modifier: " + mcmStr(annotation.modifier))

    ' the datum reference frame

```

```
strs.Add ("")
Call DatumsStr(annotation)
' the direction of the planar zone
strs.Add ("")
boolstatus = annotation.GetZoneDirection(I, J, K)
strs.Add ("Planar Zone Direction: " + FormatVector(I, J, K))
```

End Sub

```
Private Sub ConcentricityData(annotation As DimXpertConcentricityTolerance)
```

```
    strs.Add ("")
    strs.Add ("Concentricity Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the material condition modifier applied to feature
    strs.Add (" Modifier: " + mcmStr(annotation.modifier))

    ' the datum reference frame
    Call DatumsStr(annotation)
```

End Sub

```
Private Sub TotalRunoutData(annotation As DimXpertTolerance)
```

```
    strs.Add ("")
    strs.Add ("Total Runout Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the datum reference frame
    Call DatumsStr(annotation)
```

End Sub

```
Private Sub CircularRunoutData(annotation As DimXpertTolerance)
```

```
    strs.Add ("")
    strs.Add ("Circular Runout Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the datum reference frame
    Call DatumsStr(annotation)
```

End Sub

```

Private Sub LineProfileData(annotation As DimXpertLineProfileTolerance)
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean

    strs.Add ("")
    strs.Add ("Line Profile Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the outer (outside material) tolerance value
    strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))

    ' the vector normal to the profile zones
    strs.Add ("")
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add ("Planar Zone Vector: " + FormatVector(I, J, K))

    ' the datum reference frame
    Call DatumsStr(annotation)

End Sub

Private Sub CompositeLineProfileData(annotation As DimXpertCompositeLineProfileTolerance)

    strs.Add ("")
    strs.Add ("Composite Line Profile Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the outer (outside material) tolerance value
    strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))

    ' the vector normal to the profile zones
    strs.Add ("")
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add ("Planar Zone Vector: " + FormatVector(I, J, K))

    ' the datum reference frame
    Call DatumsStr(annotation)

    ' the datum reference frame for the orientation and form
    strs.Add ("Composite Datums:")
    strs.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    strs.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))

```

```

        strs.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))

End Sub

Private Sub SurfaceProfileData(annotation As DimXpertSurfaceProfileTolerance)

    strs.Add ("")
    strs.Add ("Surface Profile Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the outer (outside material) tolerance value
    strs.Add (" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))

    ' the datum reference frame
    Call DatumsStr(annotation)

End Sub

Private Sub CompositeSurfaceProfileData(annotation As
DimXpertCompositeSurfaceProfileTolerance)

    strs.Add ("")
    strs.Add ("Composite Surface Profile Tolerance Compartment")

    ' the tolerance value
    strs.Add (" Tolerance Upper Tier: " + FormatDouble(annotation.Tolerance))

    ' the outer tolerance value
    strs.Add (" Outer Tolerance Upper Tier: " +
FormatDouble(annotation.OuterToleranceValue))

    ' the datum reference frame for the location
    Call DatumsStr(annotation)

    ' the datum reference frame for the orientation and form
    strs.Add ("Composite Datums:")
    strs.Add (" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    strs.Add (" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True", "False"))
    strs.Add (" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True", "False"))
End Sub

Private Sub OrientationData(annotation As DimXpertOrientationTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim annoType As Long
    Dim boolstatus As Boolean

```

```

annoType = annotation.Type
' the type or orientation tolerance
If annoType = swDimXpertGeoTol_Perpendicularity Then
    strs.Add ("Orientation Type: Perpendicularity")
ElseIf annoType = swDimXpertGeoTol_Parallelism Then
    strs.Add ("Orientation Type: Parallelism")
ElseIf annoType = swDimXpertGeoTol_Angularity Then
    strs.Add ("Orientation Type: Angularity")
End If

strs.Add ("")
strs.Add ("Orientation Tolerance Compartment:")

' the shape of the tolerance zone
Select Case annotation.zoneType
Case swDimXpertOrientationZoneType_Cylindrical
    strs.Add (" Zone Type: Cylindrical")
Case swDimXpertOrientationZoneType_Planar
    strs.Add (" Zone Type: Planar")
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    strs.Add ("Planar Zone Vector: " + FormatVector(I, J, K))
End Select

' the tolerance value
strs.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

'material condition modifier applied to feature
strs.Add (" Modifier: " + mcmStr(annotation.modifier))

' the projected tolerance zone when specified
boolstatus = annotation.GetProjectedZone(enabled, value)
Call FormatProjectedZone(enabled, value)

' is tangent plane
strs.Add (" IsTangentPlane: " + IIf(annotation.IsTangentPlane, "True", "False"))

' the datum reference frame
Call DatumsStr(annotation)

```

End Sub

```

Private Sub FlatnessData(annotation As DimXpertFlatnessTolerance)
    Dim enabled As Boolean
    Dim length As Double, width As Double
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean

```

```

    strs.Add ("")
    strs.Add ("Flatness Tolerance Compartment:")

    ' the tolerance value
    strs.Add ("  Tolerance: " + FormatDouble(annotation.Tolerance))

    ' the per unit area data
    boolstatus = annotation.GetPerUnitArea(enabled, length, width, I, J, K)
    strs.Add ("  Per Unit Area: " + IIf(enabled, "True", "False"))
    If enabled Then
        strs.Add ("    Per Unit Length: " + FormatDouble(length))
        strs.Add ("    Per Unit Width: " + FormatDouble(width))
        strs.Add ("    Per Unit Direction: " + FormatVector(I, J, K))
    End If

End Sub

Private Sub CircularityData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Circularity Tolerance Compartment")

    ' the tolerance value
    strs.Add ("  Tolerance: " + FormatDouble(annotation.Tolerance))

End Sub

Private Sub CylindricityData(annotation As DimXpertTolerance)
    strs.Add ("")
    strs.Add ("Cylindricity Tolerance Compartment")

    ' the tolerance value
    strs.Add ("  Tolerance: " + FormatDouble(annotation.Tolerance))

End Sub

Private Sub StraightnessData(annotation As DimXpertStraightnessTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean
    Dim dist As Double
    Dim boolstatus As Boolean
    'tolerance compartment info
    strs.Add ("")
    strs.Add ("Straightness Tolerance Compartment")

    'type or shape of the tolerance zone
    Select Case annotation.zoneType
    Case swDimXpertStraightnessZoneType_Cylindrical

```

```

        str.Add (" Zone Type: Cylindrical")
Case swDimXpertStraightnessZoneType_PlanarMedian
    str.Add (" Zone Type: Planar Median")
Case swDimXpertStraightnessZoneType_Surface
    str.Add (" Zone Type: Surface Straightness")
    boolstatus = annotation.GetPlanarZoneVector(I, J, K)
    str.Add (" Zone Vector: " + FormatVector(I, J, K))
End Select

' the tolerance value
str.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))

'per unit length
boolstatus = annotation.GetPerUnitLength(enabled, dist)
str.Add (" Per Unit Length: " + IIf(enabled, "True", "False"))
If enabled Then
    str.Add (" Per Unit Length Distance: " + FormatDouble(dist))
End If

' the material condition modifier
str.Add (" Modifier: " + mcmStr(annotation.modifier))
End Sub
Private Sub ProfileData(annotation As DimXpertTolerance)
    str.Add ("")
    str.Add ("Profile Tolerance Compartment")
    ' the tolerance value
    str.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub
Private Sub SurfaceFinishData(annotation As DimXpertTolerance)
    str.Add ("")
    str.Add ("Surface Finish Tolerance Compartment")
    ' the tolerance value
    str.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub
Private Sub TangencyData(annotation As DimXpertTolerance)
    str.Add ("")
    str.Add ("Tangency Tolerance Compartment")
    ' the tolerance value
    str.Add (" Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub
Private Sub DimensionToleranceData(annotation As DimXpertDimensionTolerance)
    Dim isAngleType As Boolean
    Dim annoType As Long

```

```

Dim upper As Double, lower As Double
Dim plus As Double, minus As Double
Dim boolstatus As Boolean
annoType = annotation.Type

isAngleType = False

strs.Add ("")
strs.Add ("Dimension Tolerance Compartment")
If annoType = swDimXpertDimTol_DistanceBetween Then
    Dim distancebetween As DimXpertDistanceBetweenDimTol
    Set distancebetween = annotation
    Call DistanceBetweenData(distancebetween)
ElseIf annoType = swDimXpertDimTol_CompositeDistanceBetween Then
    Dim compdistancebetween As DimXpertCompositeDistanceBetweenDimTol
    Set compdistancebetween = annotation
    Call CompositeDistanceBetweenData(compdistancebetween)

ElseIf annoType = swDimXpertDimTol_CounterBore Then
    Dim counterBore As IDimXpertCounterBoreDimTol
    Set counterBore = annotation
    If counterBore.ReferenceFeature Is Nothing Then
        strs.Add ("Reference Feature: NULL")
    Else
        strs.Add ("Reference Feature: " + counterBore.ReferenceFeature.Name)
    End If

ElseIf annoType = swDimXpertDimTol_Depth Then
    Dim depth As IDimXpertDepthDimTol
    Set depth = annotation
    If depth.ReferenceFeature Is Nothing Then
        strs.Add ("Reference Feature: NULL")
    Else
        strs.Add ("Reference Feature: " + depth.ReferenceFeature.Name)
    End If

ElseIf annoType = swDimXpertDimTol_CounterSinkDiameter Then
    Dim countersink As IDimXpertCounterSinkDiameterDimTol
    Set countersink = annotation
    If countersink.ReferenceFeature Is Nothing Then
        strs.Add ("Reference Feature: NULL")
    Else
        strs.Add ("Reference Feature: " + countersink.ReferenceFeature.Name)
    End If

```

```

ElseIf annoType = swDimXpertDimTol_ChamferDimension Then

    Select Case annotation.ChamferType
    Case swDimXpertChamferDimensionType_Angle
        strsr.Add ("Chamfer Dimension Type: Angle")
        isAngleType = True
    Case swDimXpertChamferDimensionType_LinearDistance1
        strsr.Add ("Chamfer Dimension Type: Distance 1")
    Case swDimXpertChamferDimensionType_LinearDistance2
        strsr.Add ("Chamfer Dimension Type: Distance 2")
    End Select

ElseIf annoType = swDimXpertDimTol_AngleBetween Then
    isAngleType = True
    Dim angleBetween As IDimXpertAngleBetweenDimTol
    Set angleBetween = annotation
    ' the origin and tolerance feature
    strsr.Add ("Origin Feature: " + angleBetween.OriginFeature.Name)

    ' is supplement angle
    strsr.Add ("Supplement Angle: " + IIf(angleBetween.Supplement, "True", "False"))

ElseIf annoType = swDimXpertDimTol_CounterSinkAngle Then
    isAngleType = True
    Dim countersinkAngle As IDimXpertCounterSinkAngleDimTol
    Set countersinkAngle = annotation

    If countersinkAngle.ReferenceFeature Is Nothing Then
        strsr.Add ("Reference Feature: NULL")
    Else
        strsr.Add ("Reference Feature: " + countersinkAngle.ReferenceFeature.Name)
    End If

ElseIf annoType = swDimXpertDimTol_ConeAngle Then
    isAngleType = True

ElseIf annoType = swDimXpertDimTol_Radius Then
    Dim radiusDim As IDimXpertRadiusDimTol
    Set radiusDim = annotation
    patternTreatment = radiusDim.patternTreatment
    'If radiusDim.patternTreatment Is Nothing Then
        'strsr.Add ("Pattern Treatment: NULL")
    'End If

```

```

        If patternTreatment = swDimXpertPatternTreatmentType_unknown Then
            str.Add ("Pattern Treatment: unknown")
        ElseIf patternTreatment = swDimXpertPatternTreatmentType_CircularPattern Then
            str.Add ("Pattern Treatment: Circular pattern")
        ElseIf patternTreatment = swDimXpertPatternTreatmentType_IndividualFeatures
Then
            str.Add ("Pattern Treatment: Individual features")
        End If

```

```

    ElseIf annoType = swDimXpertDimTol_Diameter Then
        Dim diaDim As IDimXpertDiameterDimTol
        Set diaDim = annotation
        patternTreatment = diaDim.patternTreatment
        'If patternTreatment Is Nothing Then
            'str.Add ("Pattern Treatment: NULL")
        If patternTreatment = swDimXpertPatternTreatmentType_unknown Then
            str.Add ("Pattern Treatment: unknown")
        ElseIf patternTreatment = swDimXpertPatternTreatmentType_CircularPattern Then
            str.Add ("Pattern Treatment: Circular pattern")
        ElseIf patternTreatment = swDimXpertPatternTreatmentType_IndividualFeatures
Then
            str.Add ("Pattern Treatment: Individual features")
        End If

```

```

Dim indReq As Boolean
    Dim envReq As Boolean
    indReq = diaDim.IndependencyRequirement
    envReq = diaDim.EnvelopeRequirement
    str.Add ("Independency is required: " & indReq)
    str.Add ("Envelope is required: " & envReq)

```

```

End If

```

```

' conversion for radians to degrees when dimension type is angle

```

```

Dim dbl As Double

```

```

If isAngleType Then

```

```

    dbl = 57.2957795130823

```

```

Else

```

```

    dbl = 1#

```

```

End If

```

```

' the nominal, and upper and lower limits of size of the dimension

```

```

str.Add ("")

```

```

str.Add ("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue * dbl))

```

```

boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)

```

```

str.Add ("Get Upper Limit: " + FormatDouble(upper * dbl))

```

```

str.Add ("Get Lower Limit: " + FormatDouble(lower * dbl))

```

```

' the upper and lower tolerance value by type
Select Case annotation.DimensionType

Case swDimXpertDimTolType_BlockTolerance
    strs.Add ("Dimension Type: Block Tolerance")

' block tolerance
Case swDimXpertDimTolType_BlockToleranceNoNominal
    Dim blockTols As DimXpertBlockTolerances
    Set blockTols = dimXpertPart.GetBlockTolerances

    Select Case blockTols.Type
        Case swDimXpertBlockToleranceType_ASMEInch
            strs.Add ("Dimension Type: Block Tolerance No Nominal")
            If isAngleType Then
                strs.Add ("Angular Block Tolerance")
            Else
                strs.Add ("Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
            End If
        Case swDimXpertBlockToleranceType_ISO2768
            strs.Add ("Dimension Type: General Tolerance")
    End Select

Case swDimXpertDimTolType_ISOLimitsAndFits
    strs.Add ("Dimension Type: Limits and Fits")
' limits and fits tolerance
Case swDimXpertDimTolType_ISOLimitsAndFitsNoNominal
    strs.Add ("Dimension Type: Limits and Fits No Nominal")
    strs.Add ("Limits and Fits: " + annotation.LimitsAndFitsCode)
' limit dimension
Case swDimXpertDimTolType_LimitDimension
    strs.Add ("Dimension Type: Limit Dimension")
    boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
    strs.Add ("Get Upper Limit: " + FormatDouble(upper * dbl))
    strs.Add ("Get Lower Limit: " + FormatDouble(lower * dbl))
Case swDimXpertDimTolType_MAXTolerance
    strs.Add ("Dimension Type: MAXTolerance")
Case swDimXpertDimTolType_MINTolerance
    strs.Add ("Dimension Type: MINTolerance")
Case swDimXpertDimTolType_NoTolerance
    strs.Add ("Dimension Type: NoTolerance")
Case swDimXpertDimTolType_PlusMinusDimension
    strs.Add ("Dimension Type: Plus Minus Dimension")

```

' plus and minus dimension

Case swDimXpertDimTolType_PlusMinusNoNominal

 strs.Add ("Dimension Type: Plus Minus No Nominal")

 boolstatus = annotation.GetPlusAndMinusTolerance(plus, minus)

 strs.Add ("Plus Tolerance: " + FormatDouble(plus * dbl))

 strs.Add ("Minus Tolerance: " + FormatDouble(minus * dbl))

End Select

End Sub

Private Sub DistanceBetweenData(annotation As **DimXpertDistanceBetweenDimTol**)

 Dim feature As DimXpertFeature

 Dim featureFosUsage As Long

 Dim I As Double, J As Double, K As Double

 Dim boolstatus As Boolean

 ' the origin and tolerance feature along with their feature of size usage (min, max, center)

 boolstatus = annotation.**GetOriginFeature**(feature, featureFosUsage)

 strs.Add ("")

 strs.Add ("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))

 boolstatus = annotation.**GetFeature**(feature, featureFosUsage)

 strs.Add ("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))

 ' The direction vector

 boolstatus = annotation.**GetDirectionVector**(I, J, K)

 strs.Add ("")

 strs.Add ("Direction Vector: " + FormatVector(I, J, K))

End Sub

Private Sub CompositeDistanceBetweenData(annotation As **DimXpertCompositeDistanceBetweenDimTol**)

 Dim feature As DimXpertFeature

 Dim featureFosUsage As Long

 Dim plus As Double, minus As Double

 Dim I As Double, J As Double, K As Double

 Dim boolstatus As Boolean

 Dim blockTols As DimXpertBlockTolerances

 ' the origin and tolerance feature along with their feature of size usage (min, max, center)

 boolstatus = annotation.**GetOriginFeature**(feature, featureFosUsage)

 strs.Add ("")

 strs.Add ("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))

 boolstatus = annotation.**GetFeature**(feature, featureFosUsage)

 strs.Add ("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))

```

' the pattern locating feature
boolstatus = annotation.GetIntraFeature(feature, featureFosUsage)
strs.Add ("Pattern Locating Feature: " + feature.Name + " @ " +
FosUsage(featureFosUsage))

' The direction vector
boolstatus = annotation.GetDirectionVector(I, J, K)
strs.Add ("")
strs.Add ("Direction Vector: " + FormatVector(I, J, K))

' the upper and lower tolerance value for the pattern location by type

Select Case annotation.DimensionType
' plus and minus dimension
Case swDimXpertDimTolType_PlusMinusNoNominal
    strs.Add ("Pattern Locating Dimension Type: Plus Minus No Nominal")
    boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(plus, minus)
    strs.Add (" Plus Tolerance: " + FormatDouble(plus))
    strs.Add (" Minus Tolerance: " + FormatDouble(minus))
' block tolerance
Case swDimXpertDimTolType_BlockToleranceNoNominal

    Set blockTols = dimXpertPart.GetBlockTolerances

    Select Case blockTols.Type
    Case swDimXpertBlockToleranceType_ASMEInch
        strs.Add ("Pattern Locating Dimension Type: Block Tolerance No Nominal")
        strs.Add (" Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
    Case swDimXpertBlockToleranceType_IS02768
        strs.Add ("Pattern Locating Dimension Type: General Tolerance")
    End Select

End Select

' the upper and lower tolerance value for the feature to feature location by type
Select Case annotation.DimensionTypeIntraFeature
' plus and minus dimension
Case swDimXpertDimTolType_PlusMinusNoNominal
    strs.Add ("Feature Locating Dimension Type: Plus Minus No Nominal")
    boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(plus, minus)
    strs.Add (" Plus Tolerance: " + FormatDouble(plus))
    strs.Add (" Minus Tolerance: " + FormatDouble(minus))
' block tolerance

```

```
Case swDimXpertDimTolType_BlockToleranceNoNominal
```

```
Set blockTols = dimXpertPart.GetBlockTolerances
```

```
Select Case blockTols.Type
```

```
Case swDimXpertBlockToleranceType_ASMEInch
```

```
    strs.Add ("Feature locating Dimension Type: Block Tolerance No Nominal")
```

```
    strs.Add (" Block Tolerance Decimal Places: " +
```

```
Format(annotation.BlockToleranceDecimalPlacesIntraFeature, "##,##0"))
```

```
Case swDimXpertBlockToleranceType_IS02768
```

```
    strs.Add ("Feature locating Dimension Type: General Tolerance")
```

```
End Select
```

```
End Select
```

```
End Sub
```

```
Private Sub DatumsStr(annotation As DimXpertTolerance)
```

```
    strs.Add ("")
```

```
    strs.Add ("Datums:")
```

```
    Call datumStr(annotation.GetPrimaryDatums(), annotation.GetPrimaryDatumModifiers(), "Primary:")
```

```
    Call datumStr(annotation.GetSecondaryDatums(), annotation.GetSecondaryDatumModifiers(), "Secondary:")
```

```
    Call datumStr(annotation.GetTertiaryDatums(), annotation.GetTertiaryDatumModifiers(), "Tertiary:")
```

```
End Sub
```

```
Private Sub datumStr(dats As Variant, mods As Variant, datumOrder As String)
```

```
    Dim I As Long
```

```
    Dim str As String
```

```
    Dim mcm As Long
```

```
    If IsEmpty(dats) Then
```

```
        strs.Add (datumOrder + ": none")
```

```
        Exit Sub
```

```
    End If
```

```
    For I = LBound(dats) To UBound(dats)
```

```
        mcm = mods(I)
```

```
        str = str + " " + dats(I).Identifier + " @ " + mcmStr(mcm)
```

```
    Next I
```

```
    If StrComp(str, "") > 0 Then
```

```
        strs.Add (datumOrder + str)
```

```
    Else
```

```
        strs.Add (datumOrder + " <none>")
```

```
    End If
```

```
End Sub
```

```
' returns a string containing the height of the projected tolerance zone
```

```
Private Sub FormatProjectedZone(enabled As Boolean, height As Double)
```

```
    If enabled = True Then
```

```
        strs.Add ("    Projected Zone:  True")
```

```
        strs.Add ("    Zone Height: " + FormatDouble(height))
```

```
    Else
```

```
        strs.Add ("    Projected Zone:  False")
```

```
    End If
```

```
End Sub
```

```
Private Function mcmStr(mcm As Long) As String
```

```
    Dim str
```

```
    Select Case mcm
```

```
    Case swDimXpertMCM_LMC
```

```
        str = "LMC"
```

```
    Case swDimXpertMCM_MMC
```

```
        str = "MMC"
```

```
    Case swDimXpertMCM_NoMCM
```

```
        str = "NoMCM"
```

```
    Case swDimXpertMCM_RFS
```

```
        str = "RFS"
```

```
    End Select
```

```
    mcmStr = str
```

```
End Function
```

```
' returns a string containing the type of position tolerance zone used
```

```
Private Function PositionZoneType(typ As Long) As String
```

```
    Dim str As String
```

```
    Select Case typ
```

```
    Case swDimXpertPositionZoneType_CylindricalPosition
```

```
        str = "Cylindrical"
```

```
    Case swDimXpertPositionZoneType_PlanarPosition
```

```
        str = "Planar"
```

```
    Case swDimXpertPositionZoneType_SphericalPosition
```

```
        str = "Spherical"
```

```
    Case swDimXpertPositionZoneType_Boundary
```

```
        str = "Boundary"
```

```
    Case swDimXpertPositionZoneType_RadialPositionArc
```

```
        str = "RadialPositionArc"
```

```
    Case swDimXpertPositionZoneType_RadialPositionPlanar
```

```
        str = "RadialPositionPlanar"
```

```
    Case Else
```

```
        str = "N/A"
```

End Select

PositionZoneType = str

End Function

' returns a string containing the names of the SW display entities

Private Function DisplayEntity(annotation As DimXpertAnnotation) As String

Dim str As String

Dim dispEnt As Object

Dim swAnnot As SldWorks.annotation

Set dispEnt = annotation.GetDisplayEntity

If Not dispEnt Is Nothing Then

If TypeOf dispEnt Is SldWorks.annotation Then

Set swAnnot = dispEnt

str = swAnnot.GetName

End If

End If

DisplayEntity = str

End Function

' returns a string containing the feature of size usage (min, max, center)

Private Function FosUsage(value As Long) As String

Dim str

Select Case value

Case **swDimXpertDistanceFosUsage_Center**

str = "Center"

Case **swDimXpertDistanceFosUsage_MaximumSide**

str = "Max"

Case **swDimXpertDistanceFosUsage_MinimumSide**

str = "Min"

Case Else

str = "N/A"

End Select

FosUsage = str

End Function

Private Function FormatVector(I As Double, J As Double, K As Double) As String

Dim str

str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K)

FormatVector = str

End Function

Private Function FormatDouble(value As Double) As String

Dim str

```

    str = Format(value, "##,##0.0000")

    FormatDouble = str
End Function
Private Function RadiansToDegrees(value As Double) As String
    Dim str

    str = Format((value * 57.2957795130823), "##,##0.00")

    RadiansToDegrees = str
End Function
Private Function annotationTypeNameFromObject(anno As DimXpertAnnotation) As String
    annotationTypeNameFromObject = annotationTypeNameFromTypeNumber(anno.Type)
End Function
Private Function annotationTypeNameFromTypeNumber(annoTypeIndex As Long) As String
    Select Case annoTypeIndex

        Case swDimXpertDimTol_DistanceBetween
            annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
        Case swDimXpertDimTol_CounterBore
            annotationTypeNameFromTypeNumber = "CounterBore Dim"
        Case swDimXpertDimTol_Depth
            annotationTypeNameFromTypeNumber = "Depth Dim"
        Case swDimXpertDimTol_CounterSinkDiameter
            annotationTypeNameFromTypeNumber = "CounterSinkDiameter Dim"
        Case swDimXpertDimTol_ChamferDimension
            annotationTypeNameFromTypeNumber = "ChamferDimension Dim"
        Case swDimXpertDimTol_AngleBetween
            annotationTypeNameFromTypeNumber = "AngleBetween Dim"
        Case swDimXpertDimTol_CounterSinkAngle
            annotationTypeNameFromTypeNumber = "CounterSinkAngle Dim"
        Case swDimXpertDimTol_ConeAngle
            annotationTypeNameFromTypeNumber = "ConeAngle Dim"
        Case swDimXpertDimTol_Diameter
            annotationTypeNameFromTypeNumber = "Diameter Dim"
        Case swDimXpertDimTol_Length
            annotationTypeNameFromTypeNumber = "Length Dim"
        Case swDimXpertDimTol_Radius
            annotationTypeNameFromTypeNumber = "Radius Dim"
        Case swDimXpertDimTol_Width
            annotationTypeNameFromTypeNumber = "Width Dim"
        Case swDimXpertDimTol_CompositeDistanceBetween
            annotationTypeNameFromTypeNumber = "CompositeDistanceBetween Dim"
        Case swDimXpertDatum
            annotationTypeNameFromTypeNumber = "Datum"
    End Select
End Function

```

```

Case swDimXpertGeoTol_Position
    annotationTypeNameFromTypeNumber = "Position Tol"
Case swDimXpertGeoTol_CompositePosition
    annotationTypeNameFromTypeNumber = "CompositePosition Tol"
Case swDimXpertGeoTol_Symmetry
    annotationTypeNameFromTypeNumber = "Symmetry Tol"
Case swDimXpertGeoTol_Concentricity
    annotationTypeNameFromTypeNumber = "Concentricity Tol"
Case swDimXpertGeoTol_LineProfile
    annotationTypeNameFromTypeNumber = "LineProfile Tol"
Case swDimXpertGeoTol_CompositeLineProfile
    annotationTypeNameFromTypeNumber = "CompositeLineProfile Tol"
Case swDimXpertGeoTol_SurfaceProfile
    annotationTypeNameFromTypeNumber = "SurfaceProfile Tol"
Case swDimXpertGeoTol_CompositeSurfaceProfile
    annotationTypeNameFromTypeNumber = "CompositeSurfaceProfile Tol"
Case swDimXpertGeoTol_Angularity
    annotationTypeNameFromTypeNumber = "Angularity Tol"
Case swDimXpertGeoTol_Parallelism
    annotationTypeNameFromTypeNumber = "Parallelism Tol"
Case swDimXpertGeoTol_Perpendicularity
    annotationTypeNameFromTypeNumber = "Perpendicularity Tol"
Case swDimXpertGeoTol_TotalRunout
    annotationTypeNameFromTypeNumber = "TotalRunout Tol"
Case swDimXpertGeoTol_CircularRunout
    annotationTypeNameFromTypeNumber = "CircularRunout Tol"
Case swDimXpertGeoTol_Flatness
    annotationTypeNameFromTypeNumber = "Flatness Tol"
Case swDimXpertGeoTol_Circularity
    annotationTypeNameFromTypeNumber = "Circularity Tol"
Case swDimXpertGeoTol_Cylindricity
    annotationTypeNameFromTypeNumber = "Cylindricity Tol"
Case swDimXpertGeoTol_Straightness
    annotationTypeNameFromTypeNumber = "Straightness Tol"
Case swDimXpertGeoTol_Tangency
    annotationTypeNameFromTypeNumber = "Tangency Tol"
Case Else
    annotationTypeNameFromTypeNumber = "<unknown> " & CStr(annoTypeIndex)

```

```

End Select

```

```

End Function

```

Get DimXpert Tolerance2 Example (VB.NET)

This example shows how to build a part and get attributes for the following DimXpert annotations:

- * Distance-between dimension tolerance
- * Composite distance-between dimension tolerance
- * Symmetry geometric tolerance
- * Radius dimension tolerance
- * Diameter dimension tolerance

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\block.sldprt.  
' 2. Open the DimXpert toolbar from View > Toolbars.  
'  
' Create a distance-between dimension tolerance:  
' 1. Click Location Dimension on the DimXpert toolbar.  
' 2. Click the left front face of the block.  
' 3. Select Create Compound Plane from the DimXpert shortcut toolbar.  
' 4. Click the right front face of the block.  
' 5. Click the green check mark to accept the faces.  
' 6. Click the back face of the block to complete the location dimension.  
' 7. Click to position the location dimension.  
' 8. Pres Esc to end dimensioning.  
' 9. Observe the following DimXpert annotations: DistanceBetween1  
  
' Create composite distance-between and radius dimension tolerances:  
' 1. Click Auto Dimension Scheme on the DimXpert toolbar.  
' 2. Select Part type.  
' 3. Select Tolerance type.  
' 4. For Primary Datum, select the face in front of right hole.  
' 5. For Secondary Datum, select the top face of block.  
' 6. Select all feature filters.  
' 7. Click the green check mark to accept the DimXpert settings.  
' 8. Observe the following DimXpert annotations:  
'   Diameter1, DistanceBetween5, and Radius1  
  
' Create a symmetry geometric tolerance:  
' 1. Click Datum on the DimXpert toolbar.  
' 2. Click the face in front of the left hole and click  
'    to place Datum A.  
' 3. Click the green check mark to finish the Datum definition.  
' 4. Click Geometric Tolerance on the DimXpert toolbar.
```

```

' In the Geometric Tolerance Properties dialog:
' a. Select Symmetry from the Symbol dropdown.
' b. Type A for the Primary Datum.
' c. Click the face in front of the right hole.
' d. Click to place the annotation.
' e. Click OK to close the Geometric Tolerance Properties dialog.
' 5. Observe the following DimXpert annotations: Symmetry1
'
' Before running the macro:
' 1. Open the Immediate window.
' 2. Ensure that the SolidWorks.Interop.swdimxpert.dll interop
' assembly is loaded (right-click the project in the Project
' Explorer and click on Add Reference > .NET tab).
' 3. Ensure that the Microsoft Scripting Runtime library is loaded
' (right-click the project in the Project Explorer and click on Add Reference >
' COM tab).
'
' Postconditions:
' 1. Writes the output to c:\temp\dimXpertInfo.txt.
' 2. Inspect the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
' -----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swdimxpert
Imports System
Imports System.Diagnostics
Imports System.IO
Imports Scripting

Partial Class SolidWorksMacro
    Dim strs As New Collection
    Dim boolstatus As Boolean
    Dim dimXpertPart As DimXpertPart
    Dim patternTreatment As swDimXpertPatternTreatmentType_e
    Dim i As Double
    Dim j As Double
    Dim k As Double
    Sub Main()
        Dim swModelDoc As IModelDoc2
        swModelDoc = swApp.ActiveDoc
        If swModelDoc Is Nothing Then
            Exit Sub
        End If
        Dim fso As New Scripting.FileSystemObject()

```

```

Dim f As Scripting.File
Dim textStr As Scripting.TextStream
textStr = fso.CreateTextFile("c:\temp\dimXpertInfo.txt")
f = fso.GetFile("c:\temp\dimXpertInfo.txt")
'textStr = f.OpenAsTextStream(2, -2)
If textStr Is Nothing Then
    Debug.Print("Error creating temp file.")
    Exit Sub
End If
Call log("Starting DimXpert log...", textStr)
Call retrieve_info_text(swApp, textStr)
textStr.Close()
End Sub

Private Sub log(ByVal text As String, ByVal textStr As Scripting.TextStream)
    Debug.Print(text)
    textStr.WriteLine(text)
End Sub

Private Sub retrieve_info_text(ByVal swapp As SldWorks, ByVal textStr As TextStream)
    Dim dimXpertMgr As SolidWorks.Interop.sldworks.DimXpertManager
    dimXpertMgr =
swapp.IActiveDoc2.Extension.DimXpertManager(swapp.IActiveDoc2.IGetActiveConfiguration().Name,
True)

    Call log("Model: " & swapp.IActiveDoc2.GetPathName, textStr)
    Dim dimXpertPartObj As DimXpertPart
    dimXpertPartObj = dimXpertMgr.DimXpertPart
    dimXpertPart = dimXpertPartObj
    Dim vAnnotations As Object
    vAnnotations = dimXpertPart.GetAnnotations()
    Call log("-----", textStr)
    Call log("Annotations...", textStr)
    Call log("-----", textStr)
    Dim annotationTemp As DimXpertAnnotation
    Dim annotationIndex As Long
    For annotationIndex = 0 To UBound(vAnnotations)
        annotationTemp = vAnnotations(annotationIndex)
        Dim AnnotationDataText As Collection
        AnnotationDataText = AnnotationData(annotationTemp)
        Dim annotationTextIndex As Long
        For annotationTextIndex = 1 To AnnotationDataText.Count
            Call log(AnnotationDataText(annotationTextIndex), textStr)
        Next
    Next
End Sub

Public Function AnnotationData(ByVal annotation As DimXpertAnnotation) As Collection
    Dim annoType As Long

```

```

'general info
Call GeneralInfo(annotation)
annoType = annotation.Type
If annoType = swDimXpertAnnotationType_e.swDimXpertDatum Then
    Call DatumData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Position Then
    Call PositionData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition Then
    Call CompositePositionData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry Then
    Call SymmetryData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity Then
    Call ConcentricityData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile Then
    Call LineProfileData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile
Then
    Call CompositeLineProfileData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile Then
    Call SurfaceProfileData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile
Then
    Call CompositeSurfaceProfileData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity Or annoType
= swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism Or annoType =
swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity Then
    Call OrientationData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout Then
    Call TotalRunoutData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout Then
    Call CircularRunoutData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness Then
    Call FlatnessData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity Then
    Call CircularityData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity Then
    Call CylindricityData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness Then
    Call StraightnessData(annotation)
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency Then
    Call TangencyData(annotation)
Else ' any of the dimension tolerance types
    Call DimensionToleranceData(annotation)
End If
AnnotationData = str$
End Function

```

```

Private Sub Clear(ByVal str As Collection)
    str.Remove(str.Count)
    If Not str.Count = 0 Then
        Call Clear(str)
    End If
End Sub

Private Sub GeneralInfo(ByVal annotation As DimXpertAnnotation)
    Dim annoType As String
    Dim modelObj As Feature
    If Not str.Count = 0 Then
        Call Clear(str)
    End If
    str.Add("")
    str.Add("Name: " + annotation.Name)
    annoType = annotation.TypeNameFromObject(annotation)
    str.Add("Type: " + annoType)
    str.Add("Display Entity: " + DisplayEntity(annotation))
    modelObj = annotation.GetModelFeature()
    If Not (modelObj Is Nothing) Then
        str.Add("ModelFeature: " + modelObj.Name + " (" + modelObj.GetTypeName2() + ")")
    End If
End Sub

Private Sub DatumData(ByVal annotation As DimXpertDatum)
    ' the datum letter
    str.Add("")
    str.Add("Datum Letter: " + annotation.Identifier)
End Sub

Private Sub PositionData(ByVal annotation As DimXpertPositionTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean
    str.Add("")
    str.Add("Position Tolerance Compartment:")
    ' the shape of the tolerance zone
    str.Add(" Zone Type: " + PositionZoneType(annotation.ZoneType))
    ' the zone vector if the tolerance zone is planar
    If annotation.ZoneType =
swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition Then
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        str.Add(" Direction: " + FormatVector(I, J, K))
    End If
    ' the tolerance value
    str.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier applied to feature
    str.Add(" Modifier: " + mcmStr(annotation.Modifier))

```

```

        ' the projected tolerance zone when specified
        boolstatus = annotation.GetProjectedZone(enabled, value)
        Call FormatProjectedZone(enabled, value)
        ' the datum reference frame
        Call DatumsStr(annotation)
End Sub

Private Sub CompositePositionData(ByVal annotation As DimXpertCompositePositionTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean, value As Double
    Dim boolstatus As Boolean
    str.Add("")
    str.Add("Composite Position Tolerance Compartment")
    ' the shape of the tolerance zone
    str.Add(" Zone Type: " + PositionZoneType(annotation.ZoneType))
    ' the zone vector when the zone is planar
    If annotation.ZoneType =
swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition Then
        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        str.Add(" Direction: " + FormatVector(I, J, K))
    End If
    ' the tolerance value
    str.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier
    str.Add(" Modifier: " + mcmStr(annotation.Modifier))
    ' the projected tolerance zone when specified
    boolstatus = annotation.GetProjectedZone(enabled, value)
    Call FormatProjectedZone(enabled, value)
    ' the datum reference frame for the pattern location
    Call DatumsStr(annotation)
    ' the datum reference frame for the feature to feature location
    str.Add("Composite datums:")
    str.Add(" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
    str.Add(" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
    str.Add(" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
End Sub

Private Sub SymmetryData(ByVal annotation As DimXpertSymmetryTolerance)
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    str.Add("")
    str.Add("Symmetry Tolerance Compartment")
    ' the tolerance value
    str.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier applied to feature
    str.Add(" Modifier: " + mcmStr(annotation.Modifier))

```

```

        ' the datum reference frame
    strs.Add("")
    Call DatumsStr(annotation)
    ' the direction of the planar zone
    strs.Add("")
    boolstatus = annotation.GetZoneDirection(I, J, K)
    strs.Add("Planar Zone Direction: " + FormatVector(I, J, K))
End Sub

Private Sub ConcentricityData(ByVal annotation As DimXpertConcentricityTolerance)
    strs.Add("")
    strs.Add("Concentricity Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the material condition modifier applied to feature
    strs.Add("  Modifier: " + mcmStr(annotation.Modifier))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub TotalRunoutData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Total Runout Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub CircularRunoutData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Circular Runout Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the datum reference frame
    Call DatumsStr(annotation)
End Sub

Private Sub LineProfileData(ByVal annotation As DimXpertLineProfileTolerance)
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    strs.Add("")
    strs.Add("Line Profile Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the outer (outside material) tolerance value
    strs.Add("  Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
    ' the vector normal to the profile zones
    strs.Add("")

```

```

        boolstatus = annotation.GetPlanarZoneVector(I, J, K)
        strs.Add("Planar Zone Vector: " + FormatVector(I, J, K))
        ' the datum reference frame
        Call DatumsStr(annotation)
    End Sub

    Private Sub CompositeLineProfileData(ByVal annotation As
DimXpertCompositeLineProfileTolerance)
        strs.Add("")
        strs.Add("Composite Line Profile Tolerance Compartment")
        ' the tolerance value
        strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
        ' the outer (outside material) tolerance value
        strs.Add(" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
        ' the vector normal to the profile zones
        strs.Add("")
        boolstatus = annotation.GetPlanarZoneVector(i, j, k)
        strs.Add("Planar Zone Vector: " + FormatVector(i, j, k))
        ' the datum reference frame
        Call DatumsStr(annotation)
        ' the datum reference frame for the orientation and form
        strs.Add("Composite Datums:")
        strs.Add(" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
        strs.Add(" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
        strs.Add(" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
    End Sub

    Private Sub SurfaceProfileData(ByVal annotation As DimXpertSurfaceProfileTolerance)
        strs.Add("")
        strs.Add("Surface Profile Tolerance Compartment")
        ' the tolerance value
        strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
        ' the outer (outside material) tolerance value
        strs.Add(" Outer Tolerance: " + FormatDouble(annotation.OuterToleranceValue))
        ' the datum reference frame
        Call DatumsStr(annotation)
    End Sub

    Private Sub CompositeSurfaceProfileData(ByVal annotation As
DimXpertCompositeSurfaceProfileTolerance)
        strs.Add("")
        strs.Add("Composite Surface Profile Tolerance Compartment")
        ' the tolerance value
        strs.Add(" Tolerance Upper Tier: " + FormatDouble(annotation.Tolerance))
        ' the outer tolerance value
        strs.Add(" Outer Tolerance Upper Tier: " +
FormatDouble(annotation.OuterToleranceValue))

```

```

' the datum reference frame for the location
Call DatumsStr(annotation)
' the datum reference frame for the orientation and form
strs.Add("Composite Datums:")
strs.Add(" Repeat Primary: " + IIf(annotation.PrimaryInLowerTier, "True", "False"))
strs.Add(" Repeat Secondary: " + IIf(annotation.SecondaryInLowerTier, "True",
"False"))
strs.Add(" Repeat Tertiary: " + IIf(annotation.TertiaryInLowerTier, "True",
"False"))
End Sub
Private Sub OrientationData(ByVal annotation As DimXpertOrientationTolerance)
Dim I As Double, J As Double, K As Double
Dim enabled As Boolean, value As Double
Dim annoType As Long
annoType = annotation.Type
' the type or orientation tolerance
If annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity Then
strs.Add("Orientation Type: Perpendicularity")
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism Then
strs.Add("Orientation Type: Parallelism")
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity Then
strs.Add("Orientation Type: Angularity")
End If
strs.Add("")
strs.Add("Orientation Tolerance Compartment:")
' the shape of the tolerance zone
Select Case annotation.ZoneType
Case swDimXpertOrientationZoneType_e.swDimXpertOrientationZoneType_Cylindrical
strs.Add(" Zone Type: Cylindrical")
Case swDimXpertOrientationZoneType_e.swDimXpertOrientationZoneType_Planar
strs.Add(" Zone Type: Planar")
boolstatus = annotation.GetPlanarZoneVector(I, J, K)
strs.Add("Planar Zone Vector: " + FormatVector(I, J, K))
End Select
' the tolerance value
strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
'material condition modifier applied to feature
strs.Add(" Modifier: " + mcmStr(annotation.Modifier))
' the projected tolerance zone when specified
boolstatus = annotation.GetProjectedZone(enabled, value)
Call FormatProjectedZone(enabled, value)
' is tangent plane
strs.Add(" IsTangentPlane: " + IIf(annotation.IsTangentPlane, "True", "False"))
' the datum reference frame
Call DatumsStr(annotation)

```

```

End Sub

Private Sub FlatnessData(ByVal annotation As DimXpertFlatnessTolerance)
    Dim enabled As Boolean
    Dim length As Double, width As Double
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    strs.Add("")
    strs.Add("Flatness Tolerance Compartment:")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
    ' the per unit area data
    boolstatus = annotation.GetPerUnitArea(enabled, length, width, I, J, K)
    strs.Add("  Per Unit Area: " + IIf(enabled, "True", "False"))
    If enabled Then
        strs.Add("  Per Unit Length: " + FormatDouble(length))
        strs.Add("  Per Unit Width: " + FormatDouble(width))
        strs.Add("  Per Unit Direction: " + FormatVector(I, J, K))
    End If
End Sub

Private Sub CircularityData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Circularity Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub CylindricityData(ByVal annotation As DimXpertTolerance)
    strs.Add("")
    strs.Add("Cylindricity Tolerance Compartment")
    ' the tolerance value
    strs.Add("  Tolerance: " + FormatDouble(annotation.Tolerance))
End Sub

Private Sub StraightnessData(ByVal annotation As DimXpertStraightnessTolerance)
    Dim I As Double, J As Double, K As Double
    Dim enabled As Boolean
    Dim dist As Double
    Dim boolstatus As Boolean
    'tolerance compartment info
    strs.Add("")
    strs.Add("Straightness Tolerance Compartment")
    'type or shape of the tolerance zone
    Select Case annotation.ZoneType
        Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_Cylindrical
            strs.Add("  Zone Type: Cylindrical")
        Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_PlanarMedian
            strs.Add("  Zone Type: Planar Median")
    End Select
End Sub

```

```

        Case swDimXpertStraightnessZoneType_e.swDimXpertStraightnessZoneType_Surface
            strs.Add(" Zone Type: Surface Straightness")
            boolstatus = annotation.GetPlanarZoneVector(I, J, K)
            strs.Add(" Zone Vector: " + FormatVector(I, J, K))
        End Select
        ' the tolerance value
        strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
        'per unit length
        boolstatus = annotation.GetPerUnitLength(enabled, dist)
        strs.Add(" Per Unit Length: " + IIf(enabled, "True", "False"))
        If enabled Then
            strs.Add(" Per Unit Length Distance: " + FormatDouble(dist))
        End If
        ' the material condition modifier
        strs.Add(" Modifier: " + mcmStr(annotation.Modifier))
    End Sub
    Private Sub ProfileData(ByVal annotation As DimXpertTolerance)
        strs.Add("")
        strs.Add("Profile Tolerance Compartment")
        ' the tolerance value
        strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
        ' the datum reference frame
        Call DatumsStr(annotation)
    End Sub
    Private Sub SurfaceFinishData(ByVal annotation As DimXpertTolerance)
        strs.Add("")
        strs.Add("Surface Finish Tolerance Compartment")
        ' the tolerance value
        strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    End Sub
    Private Sub TangencyData(ByVal annotation As DimXpertTolerance)
        strs.Add("")
        strs.Add("Tangency Tolerance Compartment")
        ' the tolerance value
        strs.Add(" Tolerance: " + FormatDouble(annotation.Tolerance))
    End Sub
    Private Sub DimensionToleranceData(ByVal annotation As DimXpertDimensionTolerance)
        Dim isAngleType As Boolean
        Dim annoType As Long
        Dim upper As Double, lower As Double
        Dim plus As Double, minus As Double
        Dim boolstatus As Boolean
        annoType = annotation.Type
        isAngleType = False
        strs.Add("")

```

```

    str.Add("Dimension Tolerance Compartment")
    If annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween Then
        Call DistanceBetweenData(annotation)
    ElseIf annoType =
swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween Then
        Call CompositeDistanceBetweenData(annotation)
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore Then
        If annotation.ReferenceFeature Is Nothing Then
            str.Add("Reference Feature: NULL")
        Else
            str.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
        End If
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_Depth Then
        If annotation.ReferenceFeature Is Nothing Then
            str.Add("Reference Feature: NULL")
        Else
            str.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
        End If
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter
Then
        If annotation.ReferenceFeature Is Nothing Then
            str.Add("Reference Feature: NULL")
        Else
            str.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
        End If
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension Then
        Dim chamferAnno As IDimXpertChamferDimTol
        chamferAnno = annotation
        Select Case chamferAnno.ChamferType
            Case swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_Angle
                str.Add("Chamfer Dimension Type: Angle")
                isAngleType = True
            Case
swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_LinearDistance1
                str.Add("Chamfer Dimension Type: Distance 1")
            Case
swDimXpertChamferDimensionType_e.swDimXpertChamferDimensionType_LinearDistance2
                str.Add("Chamfer Dimension Type: Distance 2")
        End Select
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween Then
        isAngleType = True
        ' the origin and tolerance feature
        str.Add("Origin Feature: " + annotation.OriginFeature.Name)
        ' is supplement angle
        str.Add("Supplement Angle: " + IIf(annotation.Supplement, "True", "False"))
    ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle Then

```

```

isAngleType = True
If annotation.ReferenceFeature Is Nothing Then
    strs.Add("Reference Feature: NULL")
Else
    strs.Add("Reference Feature: " + annotation.ReferenceFeature.Name)
End If
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle Then
    isAngleType = True

ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_Radius Then
    Dim radiusDim As IDimXpertRadiusDimTol
    radiusDim = annotation
    patternTreatment = radiusDim.PatternTreatment
    'If radiusDim.patternTreatment Is Nothing Then
    'strs.Add ("Pattern Treatment: NULL")
    If patternTreatment =
swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_unknown Then
        strs.Add("Pattern Treatment: unknown")
    ElseIf patternTreatment =
swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_CircularPattern
Then
        strs.Add("Pattern Treatment: Circular pattern")
    ElseIf patternTreatment =
swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_IndividualFeatures
Then
        strs.Add("Pattern Treatment: Individual features")
    End If
ElseIf annoType = swDimXpertAnnotationType_e.swDimXpertDimTol_Diameter Then
    Dim diaDim As IDimXpertDiameterDimTol
    diaDim = annotation
    patternTreatment = diaDim.PatternTreatment
    'If patternTreatment Is Nothing Then
    'strs.Add ("Pattern Treatment: NULL")
    If patternTreatment =
swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_unknown Then
        strs.Add("Pattern Treatment: unknown")
    ElseIf patternTreatment =
swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_CircularPattern
Then
        strs.Add("Pattern Treatment: Circular pattern")
    ElseIf patternTreatment =
swDimXpertPatternTreatmentType_e.swDimXpertPatternTreatmentType_IndividualFeatures
Then
        strs.Add("Pattern Treatment: Individual features")
    End If

    Dim indReq As Boolean
    Dim envReq As Boolean
    indReq = diaDim.IndependencyRequirement

```

```

        envReq = diaDim.EnvelopeRequirement
        strs.Add("Independency is required: " & indReq)
        strs.Add("Envelope is required: " & envReq)
    End If
    ' conversion for radians to degrees when dimension type is angle
    Dim dbl As Double
    If isAngleType Then
        dbl = 57.2957795130823
    Else
        dbl = 1.0#
    End If
    ' the nominal, and upper and lower limits of size of the dimension
    strs.Add("")
    strs.Add("Compute Nominal Dimension: " + FormatDouble(annotation.GetNominalValue *
dbl))
    boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
    strs.Add("Get Upper Limit: " + FormatDouble(upper * dbl))
    strs.Add("Get Lower Limit: " + FormatDouble(lower * dbl))
    ' the upper and lower tolerance value by type
    Select Case annotation.DimensionType
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockTolerance
            strs.Add("Dimension Type: Block Tolerance")
            ' block tolerance
        Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
            Dim blockTols As DimXpertBlockTolerances
            blockTols = dimXpertPart.GetBlockTolerances
            Select Case blockTols.Type
                Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                    strs.Add("Dimension Type: Block Tolerance No Nominal")
                    If isAngleType Then
                        strs.Add("Angular Block Tolerance")
                    Else
                        strs.Add("Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
                    End If
                Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
                    strs.Add("Dimension Type: General Tolerance")
            End Select
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFits
            strs.Add("Dimension Type: Limits and Fits")
            ' limits and fits tolerance
        Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_ISOLimitsAndFitsNoNominal
            strs.Add("Dimension Type: Limits and Fits No Nominal")
            strs.Add("Limits and Fits: " + annotation.LimitsAndFitsCode)
            ' limit dimension

```

```

        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_LimitDimension
            strsr.Add("Dimension Type: Limit Dimension")
            boolstatus = annotation.GetUpperAndLowerLimit(upper, lower)
            strsr.Add("Get Upper Limit: " + FormatDouble(upper * dbl))
            strsr.Add("Get Lower Limit: " + FormatDouble(lower * dbl))
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MAXTolerance
            strsr.Add("Dimension Type: MAXTolerance")
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_MINTolerance
            strsr.Add("Dimension Type: MINTolerance")
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_NoTolerance
            strsr.Add("Dimension Type: NoTolerance")
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusDimension
            strsr.Add("Dimension Type: Plus Minus Dimension")
            ' plus and minus dimension
        Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
            strsr.Add("Dimension Type: Plus Minus No Nominal")
            boolstatus = annotation.GetPlusAndMinusTolerance(plus, minus)
            strsr.Add("Plus Tolerance: " + FormatDouble(plus * dbl))
            strsr.Add("Minus Tolerance: " + FormatDouble(minus * dbl))
    End Select
End Sub

Private Sub DistanceBetweenData(ByVal annotation As DimXpertDistanceBetweenDimTol)
    Dim feature As DimXpertFeature
    Dim featureFosUsage As Long
    Dim I As Double, J As Double, K As Double
    Dim boolstatus As Boolean
    feature = Nothing
    ' the origin and tolerance feature along with their feature of size usage (min, max,
center)
    boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
    strsr.Add("")
    strsr.Add("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    boolstatus = annotation.GetFeature(feature, featureFosUsage)
    strsr.Add("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
    ' The direction vector
    boolstatus = annotation.GetDirectionVector(I, J, K)
    strsr.Add("")
    strsr.Add("Direction Vector: " + FormatVector(I, J, K))
End Sub

Private Sub CompositeDistanceBetweenData(ByVal annotation As DimXpertCompositeDistanceBetweenDimTol)
    Dim feature As DimXpertFeature
    Dim featureFosUsage As Long
    Dim plus As Double, minus As Double
    Dim I As Double, J As Double, K As Double

```

```

Dim boolstatus As Boolean
feature = Nothing
' the origin and tolerance feature along with their feature of size usage (min, max,
center)
boolstatus = annotation.GetOriginFeature(feature, featureFosUsage)
strs.Add("")
strs.Add("Origin Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
boolstatus = annotation.GetFeature(feature, featureFosUsage)
strs.Add("Tolerance Feature: " + feature.Name + " @ " + FosUsage(featureFosUsage))
' the pattern locating feature
boolstatus = annotation.GetIntraFeature(feature, featureFosUsage)
strs.Add("Pattern Locating Feature: " + feature.Name + " @ " +
FosUsage(featureFosUsage))
' The direction vector
boolstatus = annotation.GetDirectionVector(I, J, K)
strs.Add("")
strs.Add("Direction Vector: " + FormatVector(I, J, K))
' the upper and lower tolerance value for the pattern location by type
Dim dimTol As IDimXpertDimensionTolerance
dimTol = annotation
Select Case dimTol.DimensionType
    ' plus and minus dimension
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal
        dimTol.GetPlusAndMinusTolerance(plus, minus)
        strs.Add("Pattern Locating Dimension Type: Plus Minus No Nominal")
        strs.Add(" Plus Tolerance: " + FormatDouble(plus))
        strs.Add(" Minus Tolerance: " + FormatDouble(minus))
        ' block tolerance
    Case
swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
        Dim blockTols As DimXpertBlockTolerances
        blockTols = dimXpertPart.GetBlockTolerances
        Select Case blockTols.Type
            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                strs.Add("Pattern Locating Dimension Type: Block Tolerance No
Nominal")
                strs.Add(" Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlaces, "##,##0"))
            Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
                strs.Add("Pattern Locating Dimension Type: General Tolerance")
        End Select
    End Select
End Select
' the upper and lower tolerance value for the feature to feature location by type
Select Case annotation.DimensionTypeIntraFeature
    ' plus and minus dimension
    Case swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_PlusMinusNoNominal

```

```

        strs.Add("Feature Locating Dimension Type: Plus Minus No Nominal")
        boolstatus = annotation.GetPlusAndMinusToleranceIntraFeature(plus, minus)
        strs.Add(" Plus Tolerance: " + FormatDouble(plus))
        strs.Add(" Minus Tolerance: " + FormatDouble(minus))
        ' block tolerance
    Case
        swDimXpertDimensionToleranceType_e.swDimXpertDimTolType_BlockToleranceNoNominal
            Dim blockTols As DimXpertBlockTolerances
            blockTols = dimXpertPart.GetBlockTolerances
            Select Case blockTols.Type
                Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ASMEInch
                    strs.Add("Feature locating Dimension Type: Block Tolerance No
Nominal")
                    strs.Add(" Block Tolerance Decimal Places: " +
Format(annotation.BlockToleranceDecimalPlacesIntraFeature, "##,##0"))
                Case swDimXpertBlockToleranceType_e.swDimXpertBlockToleranceType_ISO2768
                    strs.Add("Feature locating Dimension Type: General Tolerance")
            End Select
        End Select
    End Sub

    Private Sub DatumsStr(ByVal annotation As DimXpertTolerance)
        strs.Add("")
        strs.Add("Datums:")
        Call datumStr(annotation.GetPrimaryDatums(), annotation.GetPrimaryDatumModifiers(), "
Primary:")
        Call datumStr(annotation.GetSecondaryDatums(),
annotation.GetSecondaryDatumModifiers(), " Secondary:")
        Call datumStr(annotation.GetTertiaryDatums(), annotation.GetTertiaryDatumModifiers(),
" Tertiary:")
    End Sub

    Private Sub datumStr(ByVal dats As Object, ByVal mods As Object, ByVal datumOrder As
String)
        Dim I As Long
        Dim str As String
        Dim mcm As Long
        If IsNothing(dats) Then
            Exit Sub
        End If
        str = ""
        For I = LBound(dats) To UBound(dats)
            mcm = mods(I)
            str = str + " " + dats(I).Identifier + " @ " + mcmStr(mcm)
        Next I
        If StrComp(str, "") > 0 Then
            strs.Add(datumOrder + str)
        Else
            strs.Add(datumOrder + " <none>")
        End If
    End Sub

```

```

    End If
End Sub
' returns a string containing the heith of the projected tolerance zone
Private Sub FormatProjectedZone(ByVal enabled As Boolean, ByVal height As Double)
    If enabled = True Then
        strs.Add("  Projected Zone:  True")
        strs.Add("  Zone Height: " + FormatDouble(height))
    Else
        strs.Add("  Projected Zone:  False")
    End If
End Sub
Private Function mcmStr(ByVal mcm As Long) As String
    Dim str As String
    str = ""
    Select Case mcm
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_LMC
            str = "LMC"
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_MMC
            str = "MMC"
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_NoMCM
            str = "NoMCM"
        Case swDimXpertMaterialConditionModifier_e.swDimXpertMCM_RFS
            str = "RFS"
    End Select
    mcmStr = str
End Function
' returns a string containing the type of position tolerance zone used
Private Function PositionZoneType(ByVal typ As Long) As String
    Dim str As String
    Select Case typ
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_CylindricalPosition
            str = "Cylindrical"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_PlanarPosition
            str = "Planar"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_SphericalPosition
            str = "Spherical"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_Boundary
            str = "Boundary"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_RadialPositionArc
            str = "RadialPositionArc"
        Case swDimXpertPositionZoneType_e.swDimXpertPositionZoneType_RadialPositionPlanar
            str = "RadialPositionPlanar"
        Case Else
            str = "N/A"
    End Select

```

```

        PositionZoneType = str
End Function
' returns a string containing the names of the SW display entities
Private Function DisplayEntity(ByVal annotation As DimXpertAnnotation) As String
    Dim str As String
    Dim dispEnt As Object
    Dim swAnnot As SolidWorks.Interop.sldworks.Annotation
    str = ""
    'Set dispEnt = swDimXpert.GetDisplayEntity(annot)
    dispEnt = annotation.GetDisplayEntity()
    If Not dispEnt Is Nothing Then
        If TypeOf dispEnt Is SolidWorks.Interop.sldworks.Annotation Then
            swAnnot = dispEnt
            str = swAnnot.GetName
        End If
    End If
    DisplayEntity = str
End Function
' returns a string containing the feature of size usage (min, max, center)
Private Function FosUsage(ByVal value As Long) As String
    Dim str As String
    Select Case value
        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_Center
            str = "Center"
        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MaximumSide
            str = "Max"
        Case swDimXpertDistanceFosUsage_e.swDimXpertDistanceFosUsage_MinimumSide
            str = "Min"
        Case Else
            str = "N/A"
    End Select
    FosUsage = str
End Function
Private Function FormatVector(ByVal I As Double, ByVal J As Double, ByVal K As Double) As String
    Dim str As String
    str = FormatDouble(I) + ", " + FormatDouble(J) + ", " + FormatDouble(K)
    FormatVector = str
End Function
Private Function FormatDouble(ByVal value As Double) As String
    Dim str As String
    str = Format(value, "##,##0.0000")
    FormatDouble = str
End Function
Private Function RadiansToDegrees(ByVal value As Double) As String

```

```

Dim str As String
str = Format((value * 57.2957795130823), "##,##0.00")
RadiansToDegrees = str
End Function

Private Function annotationTypeNameFromObject(ByVal anno As DimXpertAnnotation) As String
    annotationTypeNameFromObject = annotationTypeNameFromTypeNumber(anno.Type)
End Function

Private Function annotationTypeNameFromTypeNumber(ByVal annoTypeIndex As Long) As String
    Select Case annoTypeIndex
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_DistanceBetween
            annotationTypeNameFromTypeNumber = "DistanceBetween Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterBore
            annotationTypeNameFromTypeNumber = "CounterBore Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_Depth
            annotationTypeNameFromTypeNumber = "Depth Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkDiameter
            annotationTypeNameFromTypeNumber = "CounterSinkDiameter Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_ChamferDimension
            annotationTypeNameFromTypeNumber = "ChamferDimension Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_AngleBetween
            annotationTypeNameFromTypeNumber = "AngleBetween Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_CounterSinkAngle
            annotationTypeNameFromTypeNumber = "CounterSinkAngle Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_ConeAngle
            annotationTypeNameFromTypeNumber = "ConeAngle Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_Diameter
            annotationTypeNameFromTypeNumber = "Diameter Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_Length
            annotationTypeNameFromTypeNumber = "Length Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_Radius
            annotationTypeNameFromTypeNumber = "Radius Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_Width
            annotationTypeNameFromTypeNumber = "Width Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDimTol_CompositeDistanceBetween
            annotationTypeNameFromTypeNumber = "CompositeDistanceBetween Dim"
        Case swDimXpertAnnotationType_e.swDimXpertDatum
            annotationTypeNameFromTypeNumber = "Datum"
        Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Position
            annotationTypeNameFromTypeNumber = "Position Tol"
        Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositePosition
            annotationTypeNameFromTypeNumber = "CompositePosition Tol"
        Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Symmetry
            annotationTypeNameFromTypeNumber = "Symmetry Tol"
        Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Concentricity
            annotationTypeNameFromTypeNumber = "Concentricity Tol"
    End Select
End Function

```

```

Case swDimXpertAnnotationType_e.swDimXpertGeoTol_LineProfile
    annotationTypeNameFromTypeNumber = "LineProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeLineProfile
    annotationTypeNameFromTypeNumber = "CompositeLineProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_SurfaceProfile
    annotationTypeNameFromTypeNumber = "SurfaceProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CompositeSurfaceProfile
    annotationTypeNameFromTypeNumber = "CompositeSurfaceProfile Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Angularity
    annotationTypeNameFromTypeNumber = "Angularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Parallelism
    annotationTypeNameFromTypeNumber = "Parallelism Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Perpendicularity
    annotationTypeNameFromTypeNumber = "Perpendicularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_TotalRunout
    annotationTypeNameFromTypeNumber = "TotalRunout Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_CircularRunout
    annotationTypeNameFromTypeNumber = "CircularRunout Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Flatness
    annotationTypeNameFromTypeNumber = "Flatness Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Circularity
    annotationTypeNameFromTypeNumber = "Circularity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Cylindricity
    annotationTypeNameFromTypeNumber = "Cylindricity Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Straightness
    annotationTypeNameFromTypeNumber = "Straightness Tol"
Case swDimXpertAnnotationType_e.swDimXpertGeoTol_Tangency
    annotationTypeNameFromTypeNumber = "Tangency Tol"
Case Else
    annotationTypeNameFromTypeNumber = "<unknown> " & CStr(annoTypeIndex)
End Select
End Function
Public swApp As SldWorks
End Class

```